FRIEDRICH-ALEXANDER
UNIVERSITÄT
ERLANGEN-NÜRNBERG
TECHNISCHE FAKULTÄT

Lehrstuhl für Informatik 3

Rechnerarchitektur

Maxim Balajan

Design and Evaluation of Partially Reconfigurable CNN Architectures on FPGAs

Bachelorarbeit im Fach Informatik

1. Dezember 2025

Please cite as:
Maxim Balajan, “Design and Evaluation of Partially Reconfigurable CNN Architectures on FPGAs,” Bachelor’s Thesis (Bachelorarbeit), University of Erlangen, Dept. of Computer Science, November 2025.

FRIEDRICH-ALEXANDER
UNIVERSITÄT
ERLANGEN-NÜRNBERG

Friedrich-Alexander-Universität Erlangen-Nürnberg
Department Informatik
Rechnerarchitektur

Martensstr. 3 · 91058 Erlangen · Germany

www3.cs.fau.de

Design and Evaluation of Partially Reconfigurable CNN Architectures on FPGAs

Bachelorarbeit im Fach Informatik

vorgelegt von

Maxim Balajan

geb. am 21. Februar 2002
in Kronach

angefertigt am

Lehrstuhl für Informatik 3
Rechnerarchitektur

Department Informatik
Friedrich-Alexander-Universität Erlangen-Nürnberg

Betreuer: **Philipp Holzinger, M. Sc.**
Betreuender Hochschullehrer: **Prof. Dr.-Ing. Dietmar Fey**

Beginn der Arbeit: **1. Juli 2025**
Abgabe der Arbeit: **1. Dezember 2025**

Erklärung

Ich versichere, dass ich die Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde.
Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Declaration

I declare that the work is entirely my own and was produced with no assistance from third parties.
I certify that the work has not been submitted in the same or any similar form for assessment to any other examining body and all references, direct and indirect, are indicated as such and have been cited accordingly.

(Maxim Balajan)
Erlangen, 1. Dezember 2025

Abstract

Convolutional Neural Networks (CNNs) are gaining more and more traction for image classification because of their high achievable accuracies. Unfortunately, standard implementations of such networks often involve computationally and memory-intensive solutions. One possible approach for tackling these problems would be the use of FPGA-based solutions, which have been extensively explored in recent years for deploying quantized CNN accelerator architectures. While *Dynamic Partial Reconfiguration (DPR)* was also investigated by a few works, performance was often inferior to purely static designs because of approaches that reconfigure large parts of the design. This work presents a partially reconfigurable architecture that calculates individual CNN layers. The design aims to keep the reconfiguration area as small as possible, improving performance compared to a fully reconfigured design. For that, the presented work outlines the abstractions needed to implement a partially reconfigurable architecture and showcases the design's high scalability and performance, achieving an overall best-case performance increase of up to ×264 compared to a C/C++ reference implementation running on an x86 architecture. This was measured by benchmarking the scalability and performance of all CNN layers of the *AlexNet* architecture in various configurations.

Contents

1 Introduction **1**

2 Related Works **5**

3 Architecture Overview **11**
3.1 Design & Dataflow 11
3.2 Control Logic 13

4 Tooling **17**
4.1 Code Generation Framework 17
4.2 Training & Inference Reference System 18

5 Implementation **21**
5.1 AXI-Stream Components 21
5.1.1 AXIS Pipeline Step 22
5.1.2 AXIS Data Width Converter 22
5.1.3 AXIS Unpacker Unit 24
5.1.4 AXIS Packer Unit 26
5.2 Configurable Slider 27
5.3 Weight Unit 29
5.4 Adder/Comparator Tree Structure 30
5.5 Temporary Layer Accumulator 32
5.6 Block & Software Integration 33

6 Evaluation **35**
6.1 Resource Utilization 37
6.1.1 Theoretical Hardware Utilization 37
6.1.2 Practical Hardware Integration 41
6.2 Performance Benchmarking 42
6.2.1 Performance Comparison of Hardware and Software 42
6.2.2 Performance Comparison of Varying Parameters 44

7 Conclusion **49**

Bibliography **55**

Chapter 1

Introduction

With the advances made in the area of *Convolutional Neural Networks (CNNs)*, it is no surprise that the field of *Computer Vision* is more relevant than ever. As a result, more and more distinct disciplines are seeking to take advantage of these new technologies whilst complying with their own specialized requirements [1].
Especially when considering different fields in need of these advances, such as autonomous and assisted driving [2], as well as medical imaging [3], the central requirements revolve around ultra-low latency and energy efficiency [4].
One trend for tackling such requirements is the use of *Application-Specific Integrated Circuits (ASICs)* due to their high performance and energy efficiency [5]. However, one huge downside of ASICs compared to other embedded chips, such as *Field-Programmable Gate Arrays (FPGAs)* or even standard *General-Purpose Processors*, is that these devices cannot be updated or reconfigured [6].
Given the rapid development of the field of CNNs, depending on a hardwired ASIC might not be ideal in the long run. For that reason, an FPGA could be more suitable to, for one, meet required performances similarly to ASICs, but also to stay flexible in the future [7].
Although FPGAs are more dynamic than ASICs, they still suffer from resource constraints when the number of logic elements on an FPGA is taken into account. This means that larger designs could not fit statically on an FPGA chip [8].

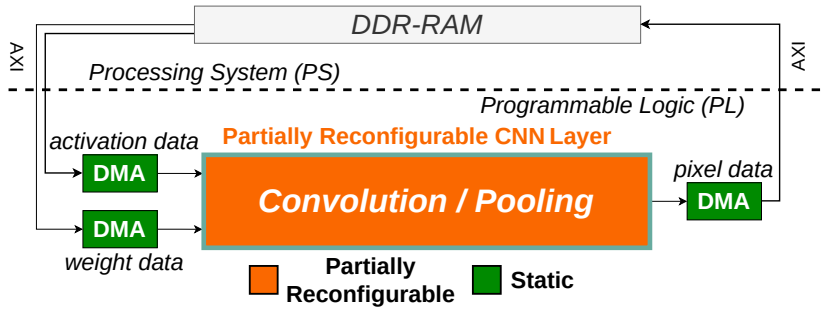


Figure 1.1 – General Accelerator Structure.

Fortunately, these devices also harbor the potential to be partially reconfigured during runtime [9]. This functionality can be seen as an opportunity to save hardware resources and, therefore, as a way to cut costs by potentially reusing existing hardware [10, 11]. In reality, such approaches unfortunately often lead to significant additional latency [12], degrading performance, when large areas are reconfigured at once [11]. This frequently is the case, because most often the majority of a design is reconfigured at once, resulting in larger chunks of the bit-stream programming the device to be rewritten, which, depending on the throughput of the architecture, can lead to the previously mentioned undesirable latencies [9].
One idea that is not really investigated in detail is how a CNN FPGA architecture could be defined so that partial reconfiguration is performed in such a way that keeps reconfiguration times to a minimum.
This work explores that idea by presenting an architecture for accelerating CNNs that is not only partially reconfigurable but also keeps the area required for reconfiguration minimal, thereby keeping reconfiguration times as small as possible. The architecture in question is illustrated in Figure 1.1.
The idea is to have an architecture that computes a single Convolution or Pooling Layer, where, after each computation, small parts of the design can be reconfigured to define new CNN layers.
Data streams are realized via a *Hardware-Software-Co-Design*, in which the *Processing System (PS)*, meaning the software, sends in the necessary activation and weight data over AXI-Stream interfaces to the *Programmable Logic (PL)*, where the data is processed. The outgoing data is streamed back from the PL to the PS.
The main focus of this work will be on presenting the abstractions needed for such an architecture and benchmarking the general functionality. Because reconfiguration times vary between different FPGA architectures, benchmarking actual reconfiguration is not the focus of this work. Moreover, this architecture is completely focused on *Convolution* and *Pooling* Layers. This is for one because of their central roles in CNN architectures, and because other phases, such as *Fully-Connected* Layers, requiring completely different architectures to begin with.

In Chapter 2 of this work, a few related works will be presented on how various CNN architectures are already realized in the ASIC and FPGA domains. Chapter 3 will outline the general dataflow of the newly presented architecture and describe the unit's behavior from a broader perspective. This is followed by Chapter 4, which discusses the various tooling and reference systems introduced to not only realize the design at large, but also to verify whether the design approach is reasonable. In Chapter 5, the individual sub-modules that make up the design are elaborated in great detail to explain the abstractions needed to implement a partially reconfigurable design. The hardware utilization and the performance of the architecture will be benchmarked in Chapter 6. The work is concluded with a small summary and by an outlook of potential future work in Chapter 7.

Chapter 2

Related Works

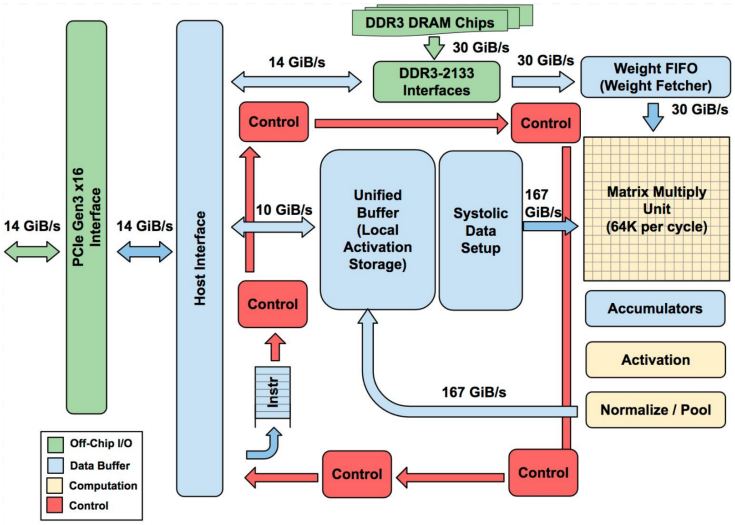


Figure 2.1 – Block Design of Google's *Tensor Processing Unit* [13].

As previously mentioned in the introduction, state-of-the-art approaches for implementing CNN architectures with low latency and power consumption use static architectures realized either on customized ASICs or on FPGA-based designs.
One unique ASIC architecture that has shown promising results is Google's *Tensor Processing Unit (TPU)* [13], depicted in Figure 2.1. The TPU, at its core, consists of a *Matrix Multiply Unit (MXU)*, a systolic array that performs the necessary computations. Together with the *Unified Buffer*, which stores activation data in registers, and the *Weight FIFO*, the data is streamed into the MXU to compute the required CNN layers. The control logic is implemented by having the device programmed with a customized instruction set to realize different CNN layers or architectures.
The architecture achieved up to ×30 performance improvement over equivalent GPU or CPU implementations, as well as a significant decrease in energy consumption.

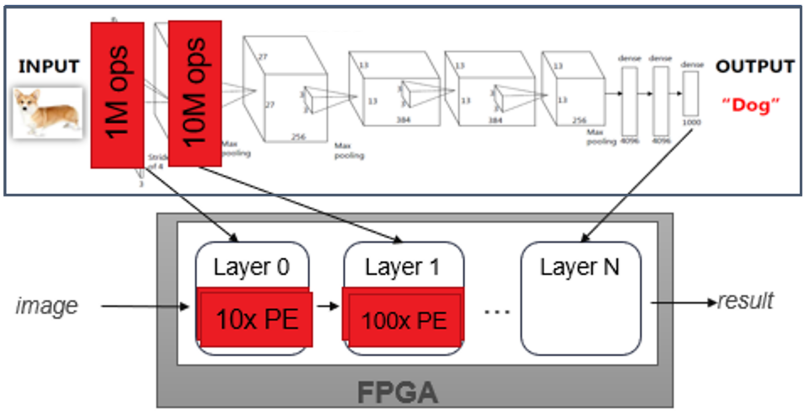


Figure 2.2 – Example of the *FINNs* end-to-end flow [14].

Despite the architecture being programmable, it does not offer the same level of flexibility as the other solutions mentioned previously. This means that, depending on the use case, more performance or energy could be saved by considering even more specialized architectures, as it is possible on other specialized ASICs or with FPGA-based designs.

One work that explores different, more specialized architectures is that of S. Ghaffari and S. Sharifian [15]. In their work, two different *High Level Synthesis (HLS)* designs were proposed, the first architecture being a data flow architecture and the second one a design only providing hardware for each layer type, where each layer is controlled by a global control loop unit, which also manages the data transfers between layers. Although a more general-purpose architecture was explored, both designs remain static and constrained by FPGA resource limitations, because neither approach utilizes *Dynamic Partial Reconfiguration (DPR)* to achieve greater flexibility.

Another solution for static dataflow designs, which, unlike the previous work, can be prototyped much more efficiently, is the *FINN* compiler from researchers at AMD [14]. *FINN* is a machine learning framework that provides the end-to-end flow depicted in Figure 2.2. As seen in the figure, it first takes in a quantized CNN architecture, which is trained using *Brevitas* [16], a quantization framework based on *PyTorch* [17]. This framework also allows control over the quantization granularity for each layer. These layers are then transformed into custom accelerator designs, using various HLS or RTL IP cores provided by the compiler. These layers are then connected via AXI-Stream to create a highly customized dataflow architecture with low latencies. The work by Quentin Ducasse et al. benchmarks the framework across various activation and weight precisions, by comparing resulting accuracies, resource requirements, and overall performance, providing a test suite for finding optimal parameters for a specific architecture [18].

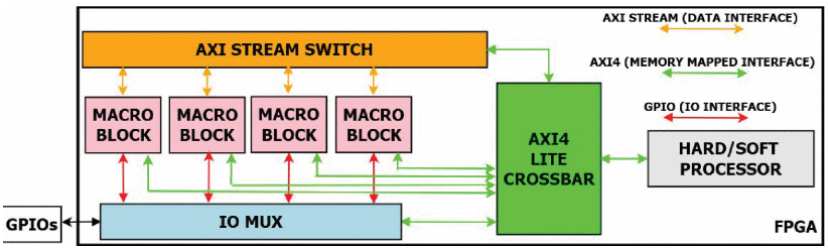


Figure 2.3 – *Macro Block* based design with dynamic interconnections. [10]

Although *FINN* goes into a much more dynamic direction, the resulting designs are still completely static, meaning the accelerators are not designed with partial reconfiguration in mind.

The approach chosen by M. Ghasemi and Y. Yang, on the other hand, presents an actual partially reconfigurable architecture even tailored towards *Deep Neural Networks (DNNs)* [11]. This design adopts ResNet-18 [19] as the base model and is structured into three reconfigurable regions, consisting of multiple chained convolution phases. These reconfigurable IPs are configured by separate reconfiguration units that load the corresponding partial bitstreams into these regions. All layers are connected to a static pooling unit for further computation.

The work also provides actual reconfiguration times that the authors measured. The evaluation platform used was the *Xilinx Zynq-7000 ZC7020*, where for the individual IP cores a configuration time of $\approx 40ms$ was achieved. This means that the unit could very likely achieve better reconfiguration performances if the design were ported to, e.g., an UltraScale+ MPSoC device using a configuration mode with better bandwidth than the ones provided for the 7000 SoCs [9]. Although the design was built with DNNs in mind, the authors leave the evaluation of such architectures as future work.

Unfortunately, the work also did not explore the possibility of reconfiguring only small parts of a CNN layer, meaning that only a whole or multiple layers could be reconfigured in a single configuration cycle, further increasing the reconfiguration delay reported by the authors.

A similar, but more fine-grained, approach utilizing DPR to accelerate CNN architectures was proposed by Hasan Irmak et al. [10]. The work illustrates a dynamic, reconfigurable LeNet-5 [20] architecture, depicted in Figure 2.3, consisting of *Macro Blocks* marked as reconfigurable *Pblock* areas that compute individual layers. These blocks are controlled via an *AXI4 Lite Crossbar* where the in- and outgoing data streams are multiplexed to the PS.

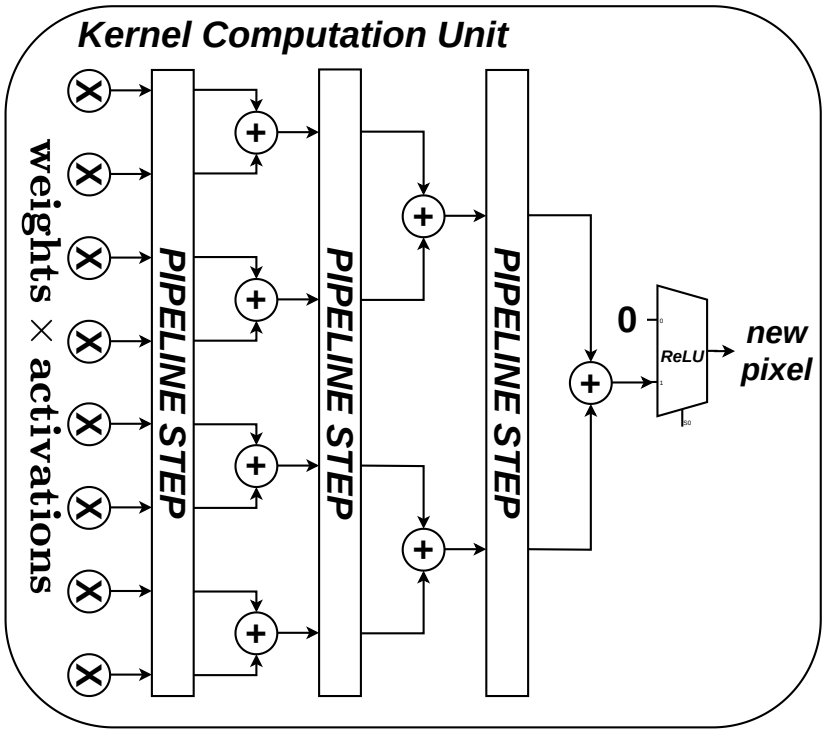


Figure 2.4 – Schematic of the *Kernel Computation Unit* [21].

Compared to previously achieved results with dynamic LeNet-5 state-of-the-art accelerator architectures, this design utilizing DPR reduces the inference time of a single image by nearly a factor of three, having an inference time of $\approx 63\mu s$. This work only covers the scalability of the hardware design for the LeNet-5 architecture as well as similar ones, but does not investigate larger CNNs.
Also, similar to previous work on partial reconfiguration, the design supports only single-layer reconfiguration, leaving much room for optimization again when considering hierarchical, more fine-granular reconfiguration.
Besides all the other works mentioned previously, this work in particular is heavily influenced by the static but customized CNN accelerator that was developed in a previous work of mine together with Markus E. Gerber, being a static LeNet-5 accelerator, that was implemented in the course *Rechnerarchitekturen für Deep-Learning Anwendungen (RADL)* [21]. This work presents a static dataflow architecture that was fine-tuned and strongly adapted to the LeNet-5 CNN architecture.
Because all the weight data could be stored inside of BRAM tiles, this work achieved a super low latency of $\approx 22\mu s$ for single image inference. Although the work presented in this thesis can be seen as a continuation, most hardware modules implemented in RADL could not be used in this work because of their static nature.

The abstractions to be seen throughout Chapter 5 led to entirely new implementations of submodules, such as the *Line Buffer* structure and adder/comparator structures, as well as many other designs within the LeNet-5 accelerator. The only major component carried over for this work, being used throughout the design, or rather throughout the adder/comparator trees, is illustrated in Figure 2.4.
The *Kernel Computation Unit* is a static, fully pipelined tree structure that can be fine-tuned by setting a variety of generics. This unit is instantiated for each core inside an adder/comparator bundle (compare Section 5.4).
As seen with all the previous work, improving performance as well as the flexibility of CNN hardware designs have already been thoroughly investigated, even using DPR as an approach. What was not done before is the attempt to create a partially reconfigurable architecture that allows the user to control the granularity of reconfiguration, so that only small regions of the FPGA need to be reconfigured when calculating an entire CNN architecture. How this can be achieved will be explored further in the upcoming chapters.

Chapter 3

Architecture Overview

Before taking a deep dive into the implementation specifics of the accelerator design and the tooling needed to verify and implement it, a general overview of the design will be provided in Section 3.1. After that, the global control logic will be discussed in greater detail in Section 3.2 to gain an understanding of how the unit implements actual CNN Layers. After providing a general overview, the tooling and the specifics of the individual components, together with their integration with the software, will be further explained in Chapter 4 and Chapter 5, respectively.

3.1 Design & Dataflow

Figure 3.1 illustrates the design, which implements a generic, partially reconfigurable architecture to compute individual Convolution- and Pooling-Layers. The core idea of the design is fairly similar to the main design of the $Conv_Seq$ unit presented in a previous work of mine (RADL) [21].
The design calculates one output channel per iteration, but, unlike the previous work, it is strongly tailored to support partial reconfigurability and therefore to be reusable as much as possible.
As a result, the unit falls somewhere between a dataflow architecture and a more general accelerator design. For one, it implements a dataflow for calculating single output channels, but its iterative nature prevents it from pipelining data across layers until the unit finishes its computation.

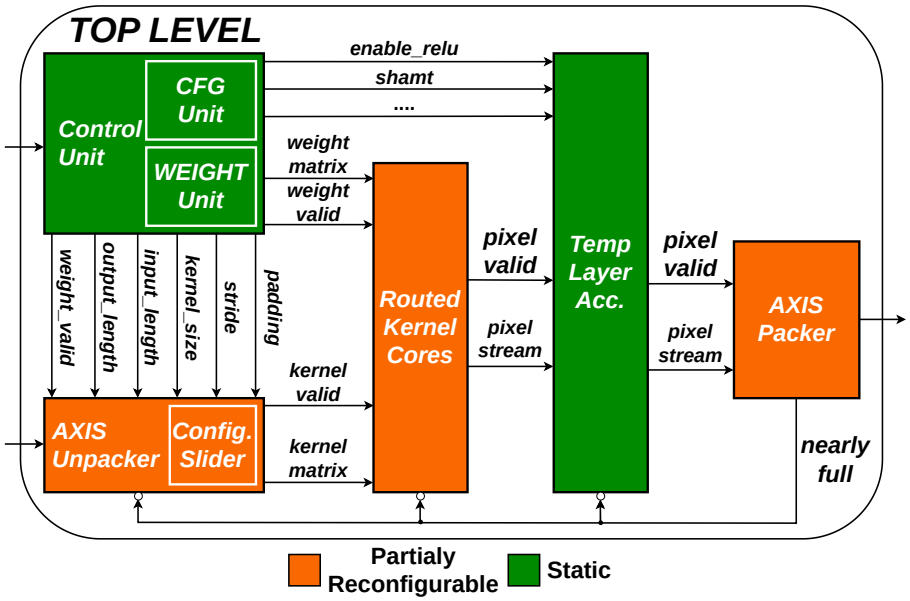


Figure 3.1 – Proposed Accelerator Design for generic CNN Layers.

First, the pixel data for one or more channels, together with their corresponding weight values, is streamed over two separate AXI-Stream channels from the software via DMA directly into the design.

The weight data values are indexed into a static register memory region provided by the *Weight Unit* (compare Section 5.3) embedded inside the global *Control Unit*, which manages the iterations of all the CNN layers.

The pixel data, however, is first converted from an AXI-Stream to a Pixel-by-Pixel stream needed by the design, which is achieved by the *AXIS Unpacker Unit* (compare Section 5.1.3). The pixels are then streamed one at a time into the *Configurable Slider*, a partially reconfigurable *Line Buffer* structure that serves to retrieve the necessary kernels for the convolution and pooling phases. This particular structure will be further elaborated in Section 5.2.

When both the weight memory and the first kernel windows are initialized, the unit starts sending the kernel windows together with their corresponding weights to the *Routed Kernel Cores* unit, a pipelined tree structure consisting of adders or comparators depending on the layer, implemented such that it supports newly generated tree structures when partially reconfigured.

To tackle these requirements, the necessary tooling for the generation of hardware modules will be presented in Section 4.1, whereas Section 5.4 will go into greater detail on the design abstractions needed for this central core.

After the desired channels have been computed, the newly calculated pixel streams are passed through the static *Temporary Layer Accumulator*, which is only utilized during a convolution phase. Because calculating an entire output channel can often result in designs too large to be placeable in the PL, a single output channel can require multiple iterations to retrieve its data.
For that reason, the temporary output channels need to be accumulated after each iteration. Only during the last iteration of a single output channel will the data be streamed to the next phase. A further explanation on how exactly the accumulator is implemented can be found in Section 5.5.
When the accumulator is passing on all pixel stream data, the data is then stored in the *AXIS Packer Unit*, where the individual pixel data is packed into AXI-Stream packages, stored in BRAM FIFOs, and retrieved when the software performs a read transaction. More information on this unit, including its stalling behavior, already hinted at in Figure 3.1, can be found in Section 5.1.4.

3.2 Control Logic

As already briefly touched upon in Section 3.1, the actual computational unit can often only realize smaller parts of a convolution or pooling phase because of hardware limitations. When considering this, the unit needs to operate iteratively, where each sub-iteration computes a temporary output channel that is accumulated and quantized by the *Temporary Layer Accumulator* (compare Section 5.5).
Figure 3.2 illustrates the simplified behavior of the global control logic, showing how the hardware is being enabled to compute entire CNN layers iteratively. When the accelerator is reset at the beginning, it enters the *Configuration Retrieval* state. In this state, the unit expects to receive a configuration instruction that programs certain parts of the hardware in form of registers configuring the device. These registers are stored inside the *Configuration Unit* embedded in the global *Control Unit*. The instruction, depicted in Figure 3.3, is sent over the AXI-Stream interface used for the weight data before the software sends any weight or pixel data. This instruction contains various values, e.g., the input and output channel sizes, which determine how many iterations are needed to compute an entire layer. It also contains information such as the image input and output lengths, as well as other values such as the kernel size, padding size, and stride length, which control the behavior of the generic line buffer structure (compare Section 5.2).

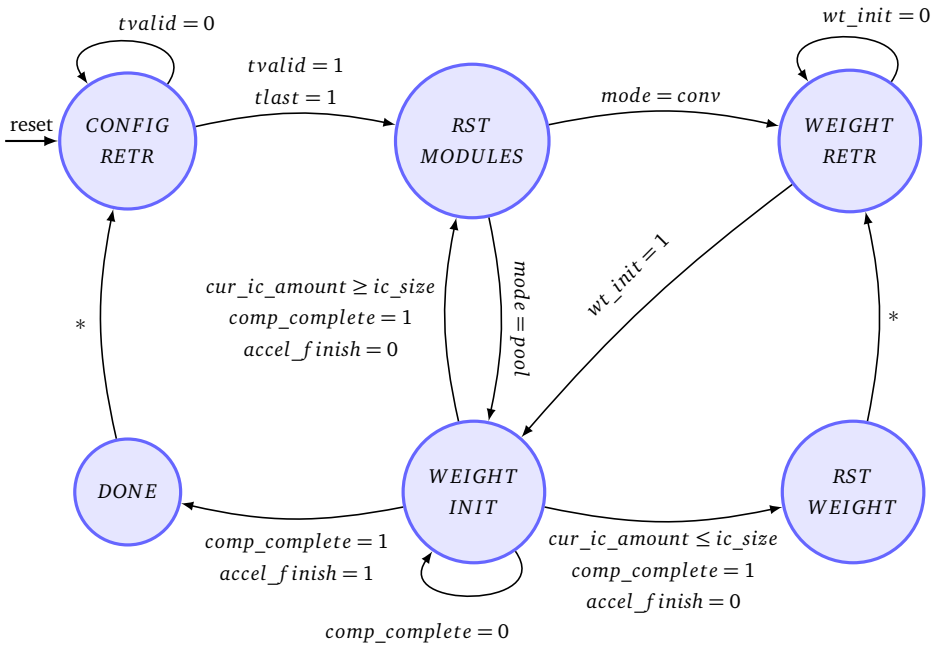


Figure 3.2 – Simplified Moore FSM representation of the Control Logic.

At first glance, this might seem relatively inefficient to require an extra read transaction, and it begs the question of why the configuration data is not hard-coded as a Look-Up-Table inside the design itself, especially if other parts are already required to be reconfigured. The reasoning for this design decision lies in the fact that various CNN architectures have a few subsequent layers fairly similar to each other, e.g., such as the last three Convolution Layers inside the *AlexNet* [22] architecture (compare Section 6.2.1). All three layers compute convolutions with 3×3 kernels, meaning that all major structures can remain the same. With that, if the configuration data is streamed to the unit, in such cases, no reconfiguration needs to be performed at all. Whether this approach is really better than having the registers hard-coded definitely depends on the FPGA architecture, its reconfiguration time as well as on the performance of the AXI read transaction.

After the configuration is received, all sub-modules get reset with the *Reset Modules* state so that the configuration data programs the modules in question. Depending on the mode, meaning if the computation will be a convolution or a pooling phase, the state machine will transition differently. When computing a convolution phase, the unit changes to the *Weight Retrieval* state, during which the unit requests the weight data and, as long as the *Line Buffer* structure of the *Configurable Slider* is not fully initialized, also the pixel data. For a more detailed overview of how the weight values are stored, see Section 5.3.

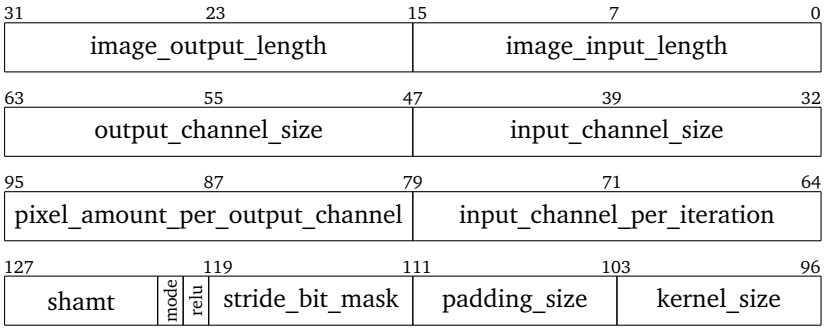


Figure 3.3 – Configuration Instruction for programming the Hardware.

When computing a pooling phase or when all weight data has been received, the unit enters the *Weight Initialized* state, which it remains in until this iteration finishes computing. Depending on whether only a temporary output channel or an entire output channel has been fully computed, the unit either resets the *Weight Unit* via the *Reset Weight* state, or it resets most of the design via *Reset Submodules*.
After all output channels have been computed and sent back successfully, the FSM transitions into the *DONE* state, where it resets the entire design and transitions back into the *Config Retrieval* state, where the unit is ready to receive a new configuration instruction for the next layer to be computed.

Chapter 4

Tooling

As already hinted at in the previous chapter, to tackle the task at hand, several tooling utilities need to be implemented first to support generic, reconfigurable units used throughout the design.
A solution to this problem is presented in Section 4.1, where a Python-based code-generation framework is introduced to create a variety of layer-specific modules.
Another aspect to consider before implementing the hardware is how neural networks typically do computations with floating-point arithmetic. Because utilizing floating point hardware components would lead to a steep increase in hardware usage, quantization in the form of mapping floating-point to integer arithmetic is explored in detail in Section 4.2 to see what type of quantization is acceptable, accuracy-wise.

4.1 Code Generation Framework

To make the generation of these partially reconfigurable architectures as streamlined as possible, tooling needs to be developed to support this endeavor. For this, a Python-based code-generation framework was implemented to create and parametrize the Top-Level Unit correctly. The framework is provided with a configuration YAML file similar to the one depicted in Figure 4.1.

```
cores:
 Nine_adder_plus_mult:
  global:
   [...]
  Nine_adder_plus_mult_rtl:
    amount         : 30
    pool_or_conv   :  1
    en_clip_adder  :  1
    en_mult_line   :  1
    input_length   :  8
    output_length  : 24
```

```
[...]
route_configs:
 global:
  [...]
 conv_5x5:
  pool_or_conv      :  1
  ic_per_iteration  :  8
  oc_per_iteration  :  1
  input_amount      : 25
  padding           :  2
  stride            :  0
```

Figure 4.1 – Snippet of example YAML Configuration.

First, the config expects a *Cores* section that defines adder or comparator trees, which can be used to create the trees mentioned in Section 3.1. These cores are then consumed by the framework to route them logically and to create actual adder or comparator trees. How the generated hardware is abstracted to support partial reconfiguration is further elaborated in Section 5.4.
Finally, the framework also generates a VHDL-'93 wrapper for the Top Level, setting all important, layer-specific generics. This framework also has the potential to be expanded, allowing the fine-tuning of the design at hand to be configured even further, which will be explored in Chapter 7.
With that, a new layer structure can easily and, more importantly, rapidly be created for the prompt employment of the CNN layer at hand.

4.2 Training & Inference Reference System

One important factor when constructing a hardware design implementing convolution and pooling phases is that traditional floating-point arithmetic present in such architectures often leads to significant hardware and latency overhead. For that reason, quantization of such networks, mapping floating-point arithmetic to integer arithmetic in the form of fixed-point numbers, can be seen as a solution to such a problem. One thing to keep in mind, though, when quantizing a network, is to ensure that the accuracy of a CNN architecture is not significantly reduced when using reduced-precision activation and weight data.

Table 4.1 – Comparison between different *CIFAR-10* inference approaches.

Network	Implementation	Data-Type	Epochs	Accuracy
ResNet	Deep Residual Learning for Image Recognition [19, 23]	*float32*	N/A	93.57%
AlexNet	ImageNet Classification with Deep CNNs [22, 23]	*float32*	N/A	83.00%
AlexNet	Efficient Quantization and HW Implementation of AlexNet on Resource-limited FPGAs [24]	*int8*	250	83.16%
AlexNet	*Part. Reconfigurable Architecture (Python and C/C++)*	*int8*	*100*	*76.67%*
AlexNet	Efficient Quantization and HW Implementation of AlexNet on Resource-limited FPGAs [24]	*int1*	250	72.66%

The training to verify the applicability of such a quantized design was performed using *PyTorch* [17] together with the previously mentioned quantization framework of AMD called *Brevitas* [16]. For quantization, an 8-bit quantization was chosen for both the activation and weight data to keep hardware resources minimal whilst still retaining accuracy. Also, no biases were used inside the network.
The network of choice for benchmarking these accuracies will be the *AlexNet* architecture [22]. The architecture will be explored further in Chapter 6. For now, it is only important to note that, with this architecture, it is possible to classify images provided by the *CIFAR-10* dataset [25]. This dataset consists of 60.000 32×32 color images, of which 50.000 are used for training. Trained for over 100 epochs with the configurations mentioned above, the architecture achieves an accuracy of $\approx 76.67\%$ on the 10.000 test images. This accuracy was also achieved utilizing a naive C/C++ inference framework introduced in a previous work of mine (RADL) that utilizes specialized data types such as 24-bit fixed-point integers to resemble the simulated hardware closer, but also updated to include new necessary feature sets, such as variable stride lengths for convolutions as well as padding of input channels [21].
The main question is how the accuracy compares with that reported in other studies. Section 4.2 compares the approach described above with other results on the *CIFAR-10* dataset across various factors. One of the highest accuracies achieved was by Kaiming He et al., which did not utilize AlexNet, but rather residual learning to increase the accuracy to $\approx 94\%$ [19], which is a much higher increase when compared to the original accuracy achieved by Alex Krizhevsky et al. [22] of $\approx 83\%$. It should be noted that this comparison must be made carefully, because the work by A. Krizhevsky et al. was based on the *ImageNet* dataset, whereas the work by Ying Bi et al. claims to achieve this accuracy also on the *CIFAR-10* dataset [23].

Nonetheless, because this work accelerates generic CNN layers, this design could also be implemented within any CNN architecture to accelerate certain layers; however, more research is needed to assess the performance impact on other architectures.
The work by E. Rex and X. Wang on quantized hardware implementations of the AlexNet architecture using *FINN* achieves even higher accuracies with the same 8-bit quantization [24], but is trained for more than twice as many epochs. Future work could further investigate how the network could be improved to speed up training and increase accuracy (see Chapter 7).
One optimization already found was when grouping input channels for convolution phases. This significantly reduced training time, taking only roughly 20 epochs to reach the same accuracy reported in Section 4.2.
All in all, the accuracy is good enough for working on an architecture with such quantization metrics in mind.

Chapter 5

Implementation

The following chapter provides a detailed overview of the implementation specifics that comprise the design. First, the AXI-Stream interfaces and their corresponding sub-components will be explained in Section 5.1 to get a better understanding of how the unit communicates to the outside world. Following that, Section 5.2 goes into the specifics of the *Line Buffer* structure that retrieves all valid pixel kernels. This is continued by talking about how the *Weight Unit* is realized in Section 5.3, after which the abstractions necessary for implementing a partially reconfigurable adder or comparator tree structure are explained in Section 5.4. After that, Section 5.5 will present an accumulator structure suitable for storing intermediate output channels. To conclude the chapter, Section 5.6 briefly discusses how the design is integrated with various IP cores and the software side.

5.1 AXI-Stream Components

To understand how the pixel data coming from its corresponding AXI-Stream input interface is collected and unpacked, as well as how the data is prepared for the AXI-Stream output interface, first, two integral sub-modules need to be discussed, a *AXI-Stream Pipeline Step* in Section 5.1.1 as well as a *AXI-Stream Data Width Converter* in Section 5.1.2. Following that, the actual *AXI-Stream Unpacker Unit* together with the *AXI-Stream Packer Unit* will be highlighted in Section 5.1.3 and Section 5.1.4 respectively.

5.1.1 AXIS Pipeline Step

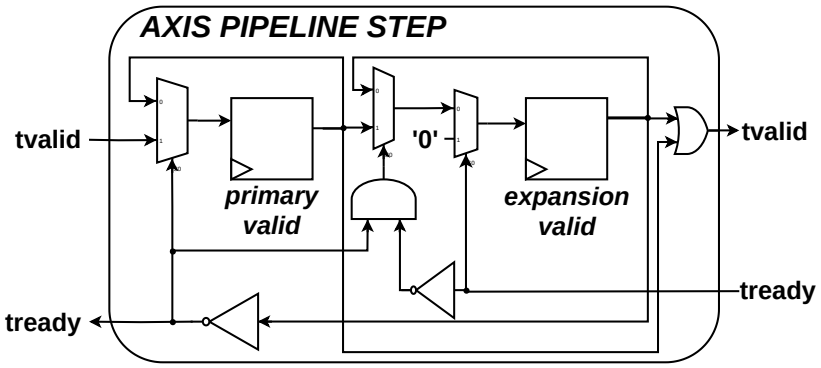


Figure 5.1 – Schematic of skid buffer for pipelining AXIS Interfaces [26].

The underlying concept of implementing a *AXIS Pipeline Step* can be seen in a simplified form in Figure 5.1 where the signals *TDATA*, *TKEEP*, and *TLAST* were, for one, left out for readability and because of their similar behavior to *TVALID*.
This component implements a simple skid buffer that registers not only the valid signal but also the ready signal. The data is first pipelined into the *primary valid* register if data can be accepted. Depending on whether the ready signal of the master interface gets de-asserted in the next cycle, the valid transaction needs to be stored in the *expansion valid* register, in order to be retrieved later when *TREADY* is set by the master. In the case, however, that the master's ready signal remains asserted, the data of the *primary valid* is used instead and multiplexed to the master's output interfaces. This multiplexing is simplified in the figure using an OR-Gate.
With this behavior, the AXI handshake is preserved while enabling improvements in timing and resource utilization when deciding to reconfigure the device (compare Section 5.1.2).

5.1.2 AXIS Data Width Converter

The *AXIS Data Width Converter* comes in two flavors: As a *Passthrough* or as a *Downsizer*. Most commonly, the unit will be instantiated as a *Passthrough*, which essentially instantiates an *AXIS Pipeline Step*. The reason for instantiating a pipeline step instead of just routing the data from the slave interface to the master interface has to do with the fact that when marked as a reconfigurable unit, the *Vivado* synthesis tool will instantiate 1-LUTs as routing switches in order to be able to reconfigure the routing [27]. Because of that, it makes more sense to instantiate a pipeline step when hardware resources will be used regardless.

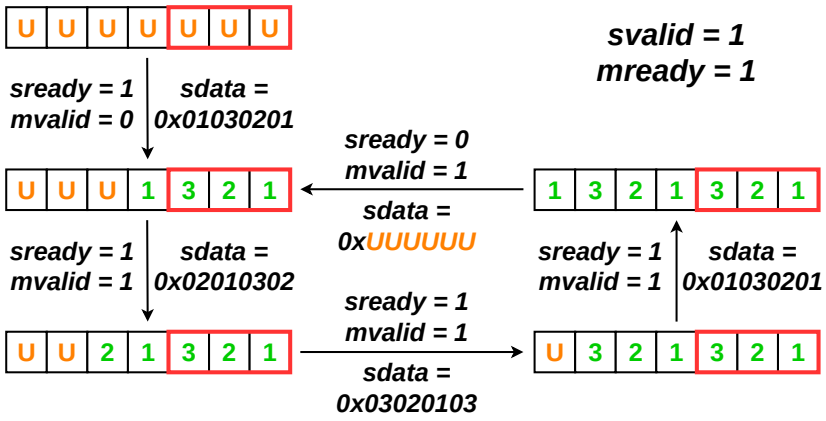


Figure 5.2 – Simplified data flow of a 4 Byte to 3 Byte *AXIS Downsizer* [28].

On the other hand, for orthodox CNN layers, a *Downsize* unit could be necessary. The reason for that is that one AXI-Stream transaction must have an equal number of packages for each concurrent channel. This edge case could occur, for example, if a phase has only three input channels, meaning that, for an AXIS bus width of 16 bytes, an extra byte would remain during this cycle.
One approach would be to pad the data sent by the software with an invalid byte, which would, of course, be highly inefficient. A preferable approach is to downsize the package size from 16 to 15 bytes while preserving their order.
Figure 5.2 shows a simplified dataflow of the downsizing unit. In this example, the *TVALID* of the slave as well as the *TREADY* of the master are always asserted to make the behavior of the design more easily understandable.
The unit has a central barrel shifter with the combined size of the input and output byte widths, resulting in a buffer length of 7 Bytes for the illustrated example. At the beginning, when the buffer is empty, the unit sets *TREADY* of the slave to high to start receiving input data, but also sets *TVALID* of the master to low because no data can be sent yet. After that cycle, the master always has enough packages to consume, meaning for the next three cycles the unit will always send the first three entries to the master and insert the new packages into the buffer accordingly. This is realized by shifting the data out to the master and multiplexing the input stream at the index equal to the number of packages left after the master consumed the data. This process is repeated until the buffer is full, meaning it has two valid packages. If this state is reached, the slave will not accept any incoming data during the next cycle but will still be able to send a package to the master. This results in buffer space becoming available, allowing new data to be accepted into the buffer again, repeating the cycle.

5.1.3 AXIS Unpacker Unit

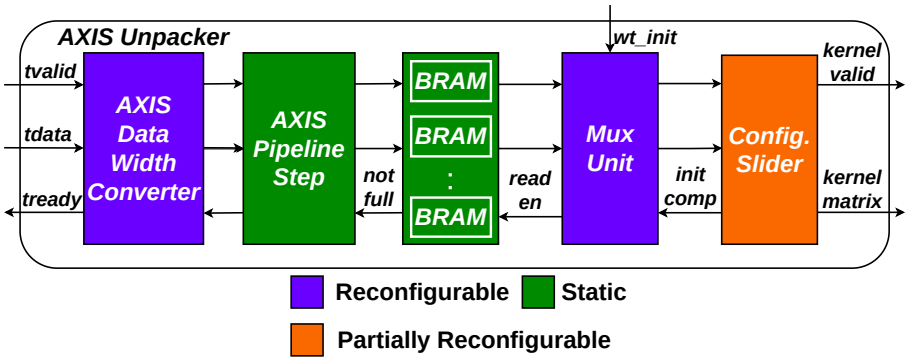


Figure 5.3 – Architecture Design of the *AXIS Unpacker Unit*.

In order to retrieve the pixel stream sent by the DMA Unit in such a way that the *Line Buffer* structure can work with the incoming data (compare Section 5.2), the packages have to be unpacked and separated into individual chunks. This is realized by the *AXIS Packer Unit* depicted in Figure 5.3.
The incoming data is first passed through a *AXIS Data Width Converter* where the incoming data is repacked into smaller packages in order to be able to distribute the data equally for each concurrent channel (compare Section 5.1.2).
Because the unit implements static repacking, it cannot be reused for subsequent CNN layers, meaning that this unit must be reconfigured when a new layer needs to be computed.
After the repacking stage, the data is pipelined for one additional cycle (compare Section 5.1.1) and then streamed into a BRAM FIFO array, where each BRAM tile is used to store one incoming byte (compare Section 5.6).
Figure 5.4 illustrates why an additional pipeline step before the BRAM array unit is necessary for improving the timing. Because BRAM tiles lie at the edge of the reconfigurable region, pipeline steps are necessary to route the incoming and outgoing data to and from the BRAM. For the writing of the BRAM tiles, this registered *write enable* is realized via the aforementioned *AXIS Pipeline Step*. The data will be written only if the data for one is valid ($tvalid = 1$) and no BRAM tile is full. If these conditions are met, the data can be written to the BRAM FIFOs without a problem.

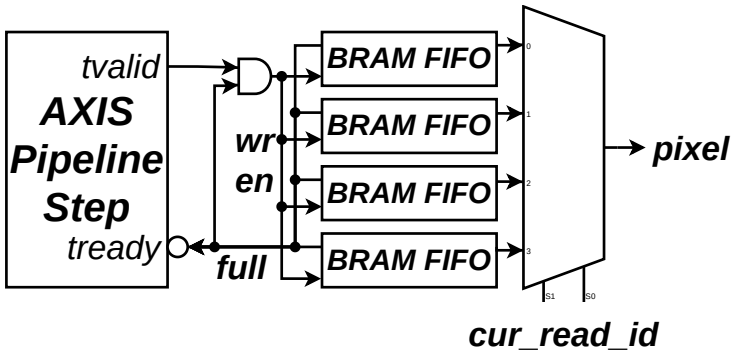


Figure 5.4 – Write enable hazards with read multiplexing (single channel).

The data reading strongly depends on the number of channels that need to be computed concurrently. The BRAM tile holding the next valid data package is determined by a static multiplexing unit that first demultiplexes the read signal onto the right FIFO and then multiplexes the data to the *Configurable Slider*, where it is used to retrieve the valid kernel windows of the activation pixels. This multiplexing is achieved by having an iterator that cycles through the BRAM tiles used for a channel. As already mentioned, all BRAM interfaces need to be pipelined, including the *read enable* signal. This, again, leads to data hazards that need to be handled. The control logic in question is depicted in Figure 5.5.

This hazard control unit uses nearly-empty signals that indicate the number of packages left in a single buffer: $nearly_empty2$ is asserted when at most two packages are left, $nearly_empty1$ when at most one is left, and $empty$ when none are present in the buffer.

As long as more than two elements are stored inside the BRAM tile, the unit is allowed to stream ($can_stream = 1$), and as long no padding is performed ($is_padding = 0$), a read transaction can be triggered. If the critical state is reached, where at most two elements are present in the buffer, the unit is locked out from starting new read transactions until this critical state is left. For this, $nearly_empty2$ has to be checked because of the two pipeline steps introduced by pipelining the empty signals as well as the *read enable*.

The final thing the unit needs is the ability to flush data to prevent deadlocks. A flush will be performed if only the last two pixel inputs are missing and both are hold by the buffer, as checked by the $empty$ and $nearly_empty1$ flags.

With these control mechanisms put in place, the data can be streamed to the *Configurable Slider* (compare Section 5.2) where it is processed to retrieve the desired kernel frames.

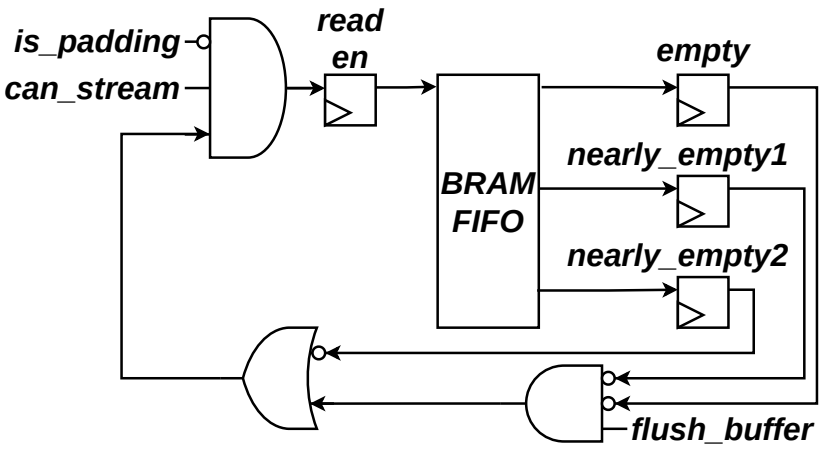


Figure 5.5 – Read enable hazard control for single BRAM FIFO.

5.1.4 AXIS Packer Unit

After the design finishes the computation of a single output channel, the resulting data needs to be packed and sent back over the AXI-Stream to the software. This is realized with the *AXIS Packer Unit* depicted in Figure 5.6.
After the output channel data has been calculated by the *Temporary Layer Accumulator* (compare Section 5.5), the pixel data must be demultiplexed onto its corresponding BRAM FIFOs. These FIFOs serve as a way to retrieve packages that need to be sent to the software via the DMA unit. When the BRAM tiles have at least one package ready, the package will be read regardless of whether the AXI-Stream has asserted *TREADY*. Because of that, the valid data waits until an asserted *TREADY* is sent to the BRAM array. After that, the next data packet is read, so it can wait to be consumed by the AXI-Stream.
This process is repeated until all the data for an output channel iteration has been received. The data is again routed over an *AXIS Pipeline Step* to improve timing to the BRAM tiles.
Now, consider the BRAM tiles growing in size and reaching a critical state where they are nearly full. In that case, the unit de-asserts a global *enable*, which is pipelined across the entire design to stall all submodules and retain the current state of the entire design. Because of that, no further hazard control is necessary for writing to the BRAM FIFO array, and because the read is always performed before the data is received, no hazard detection is needed for the read enable of the BRAM FIFOs.

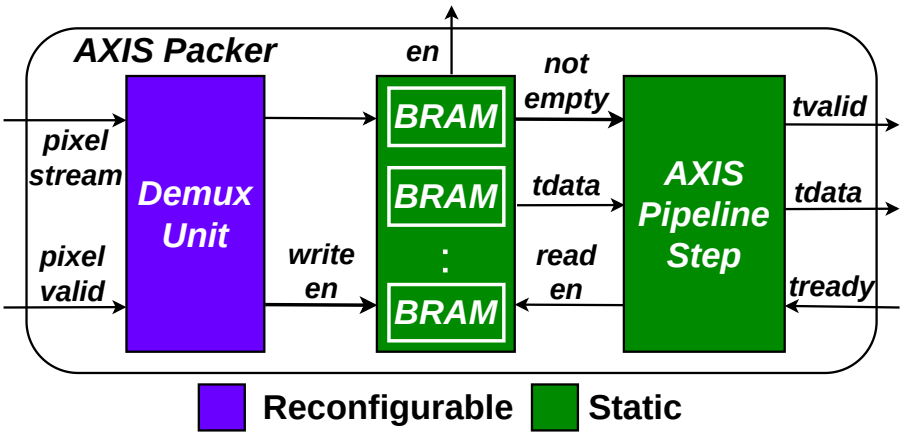


Figure 5.6 – Architecture Design of the *AXIS Packer Unit*.

5.2 Configurable Slider

To retrieve the desired kernel windows of an activation needed to compute a convolution with their corresponding weight kernels, a buffer structure must be implemented that not only satisfies this goal but also achieves the overall objective of minimizing the reconfiguration needed between layers. These objectives are realized by the *Configurable Slider* illustrated in Figure 5.7.

With classic *Line Buffer* structures as briefly touched upon in Chapter 2, the major issue is that these architectures are only suited for static FPGA designs, where the *Kernel Size* and the dimensions of the activation image are predefined. To circumvent this issue, each row of a *Line Buffer* is split into its own unit, with an updated BRAM FIFO followed by registers storing the kernel pixels.

Each BRAM FIFO can be configured with two additional signals: *is_full* and *depth*. The new *depth* signal allows the buffer to be smaller than its actual size to handle smaller image dimensions. This, of course, wastes hardware resources for layers that need smaller buffer sizes, but eliminates the need to reconfigure buffers. BRAM memory was generally kept in static regions to minimize potential timing issues that could affect meeting slack requirements.

The *is_full* signal, on the other hand, does not make the unit more dynamic, but when set during reset, the buffer is set to full, meaning the first *depth* many pixels are retrieved as 0*s*. This is achieved with additional control logic because the data in the BRAM is not resettable, so a different mechanism is needed. This signal serves to reduce a lot of initialization overhead during input data padding.

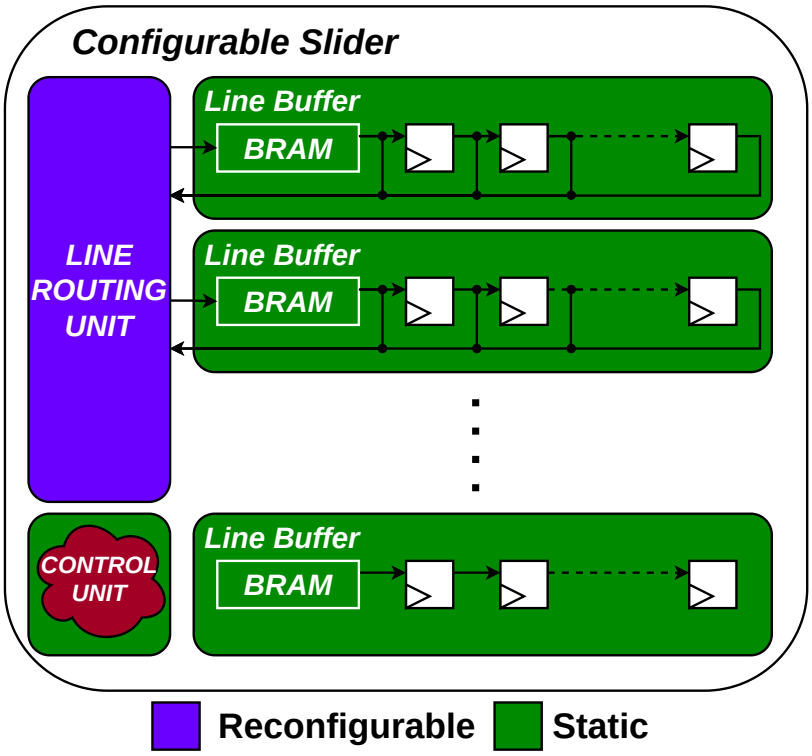


Figure 5.7 – Schematic of a partially reconfigurable *Line Buffer* Structure.

With these new functionalities in place, the resulting dynamic *Line Buffer* units need to be connected so that they realize correct kernel windows. Because dynamic multiplexing of such a routing behavior would result in significant hardware overhead, this routing behavior must be implemented statically, meaning the unit needs to be reconfigured for different layers. This is realized in the *Line Routing Unit*. This unit takes in all register values from all *Line Buffer* structures and routes them in such a way that the desired kernel windows are connected correctly.
The final piece missing is the control logic that marks a kernel as valid via the *kernel valid* signal or if all kernels have been received via the *kernel complete* signal. This module is also fully dynamic and does not need to be reconfigured, taking in current configuration values such as the stride length, padding size, and image input and output dimensions to correctly control the device.
With all these features in place, a fully generic *Line Buffer* structure can be realized with the *Configurable Slider*, thereby minimizing potential reconfiguration.

5.3 Weight Unit

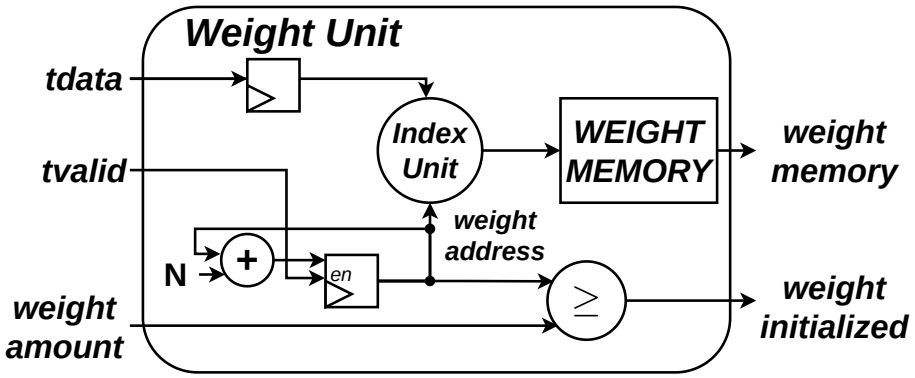


Figure 5.8 – Schematic of the *Weight Unit*.

Before computing the actual convolutions, a unit for retrieving and storing weight kernel windows must be considered. Because the data is now streamed into the unit via an AXI-Stream Interface (compare Section 3.1), the BRAM-based approach introduced in my previous work (RADL) [21] will not suffice.
Figure 5.8 illustrates the data flow on how the weight data is received and correctly written to the register memory region. Because all weight values are quantized 8-bit values, meaning every valid AXI-Stream transaction could retrieve multiple weights, a potential implementation of such a unit could be realized in two ways. Either all data is accepted in a single valid transaction cycle, or all weight values are streamed into the unit sequentially, resulting in all subsequent transactions being stalled until all the data has been consumed. Because the second approach is much slower, even though it could be implemented with much fewer hardware resources, the first idea was chosen.
This solution is implemented via an *Index Unit* that, depending on the current *weight address*, writes the new weight values to their corresponding addresses. This address is calculated using an accumulator that counts all N valid weight values received during a single transaction and adds them to the corresponding register.
When the unit has been provided with all weight values, it asserts the *weight initialized* signal in order to notify all pending units. This control signal is achieved via the incoming *weight amount* signal, which is calculated by squaring the *kernel_size* and multiplying it with the *input_channel_per_iteration* signal. Because the resulting value is a 32-bit long value, these multiplications are mapped onto one DSP slice as seen in Chapter 6. Because the *Weight Unit* is always static, this should not result in problems with routing when the design is partially reconfigured, but further investigations would be needed to prove or disprove this claim.

Although this approach works, there is still room for optimization. Having a dynamic *Index Unit* is pretty resource-intensive, so a different approach, such as a shift register with reconfigurable weight routing, would also have worked. Ultimately, the additional hardware required was acceptable, which is why this approach was chosen to reduce the reconfigurable area.
With that, a static, but dynamic weight memory unit is implemented, suitable for all possible convolution phases.

5.4 Adder/Comparator Tree Structure

In order to further process the activation kernel windows received by the *Configurable Slider* (compare Section 5.2) as well as the weight kernel windows received by the *Weight Unit* (compare Section 5.3), a unit for processing MAC or comparison operations of these windows needs to be developed.
As already discussed in Chapter 2, the *Kernel Computation Unit* could be used in a static design, but would lead to large areas that need to be reconfigured when considering partial reconfiguration. Especially when taking into account that, mainly due to convolutions, this part of the design will take up the largest share of the programmable area (compare Section 6.1.2), using this approach would lead to long latencies during reconfiguration.
To circumvent this, a different abstraction is needed so that the granularity with which the submodule is reconfigured can be precisely configured as needed.
This abstraction is illustrated in Figure 5.9, which is achieved by the *Routed Kernel Cores Unit* together with both its subcomponents, the *Routing Cores Unit* and the *Kernel Cores Unit*.
The idea behind the *Kernel Cores Unit* is to have a module that consists of both static and reconfigurable adder/comparator trees. This is achieved by grouping multiple similar tree structures into bundles, where an entire bundle is either reconfigurable or static. This abstraction is visualized in Figure 5.10.

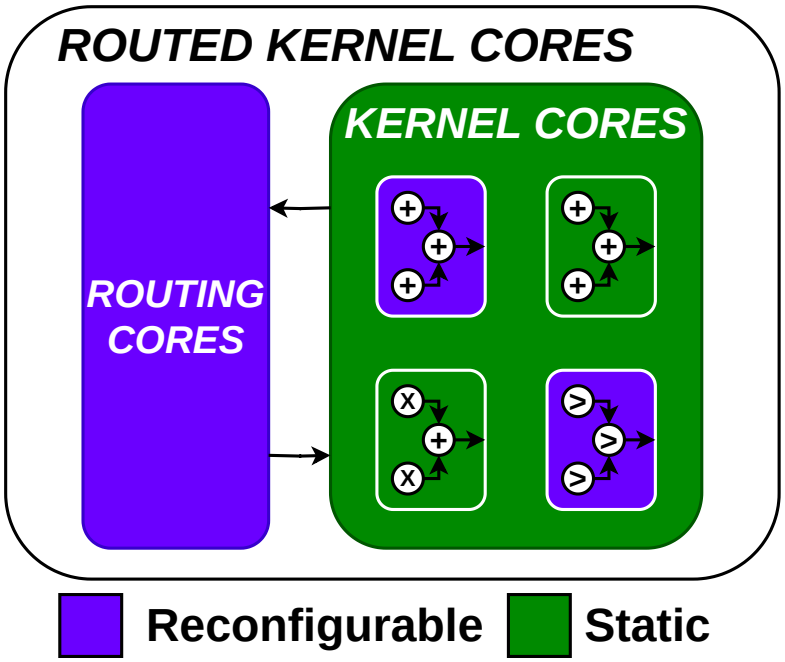


Figure 5.9 – Block Design of Adder/Comparator Tree Structure.

Two important benefits come with such an abstraction. For one, the granularity of reconfigurable areas can be tightly controlled, allowing different definitions of the abstractions to be tested when measuring the performance of reconfiguring the area occupied by the reconfigurable tree structures. It also makes it easier to define the required interfaces for these reconfigurable areas. One huge issue to consider when writing potentially reconfigurable hardware components is that the interfaces need to be exactly the same between the original component and the new components that replace it. This would not always be the case, as seen in Figure 5.10, if no global interface definition were put in place, serving as an upper bound for all possible bundle definitions.

Now with all trees put in place, the *Routing Cores Unit* solves the issue of connecting all tree structures meaningfully together. In addition to pipeline steps, this unit only implements the routing between the inputs and outputs of the *Kernel Cores Unit*. The creation of these bundles and the routing between them, with additional plausibility checks, is implemented using the framework previously introduced in Chapter 4.

With this code-generation framework, a fine-grained tree structure can be defined, enabling fine-tuning of the design to find the best-fitting configuration for the use case at hand.

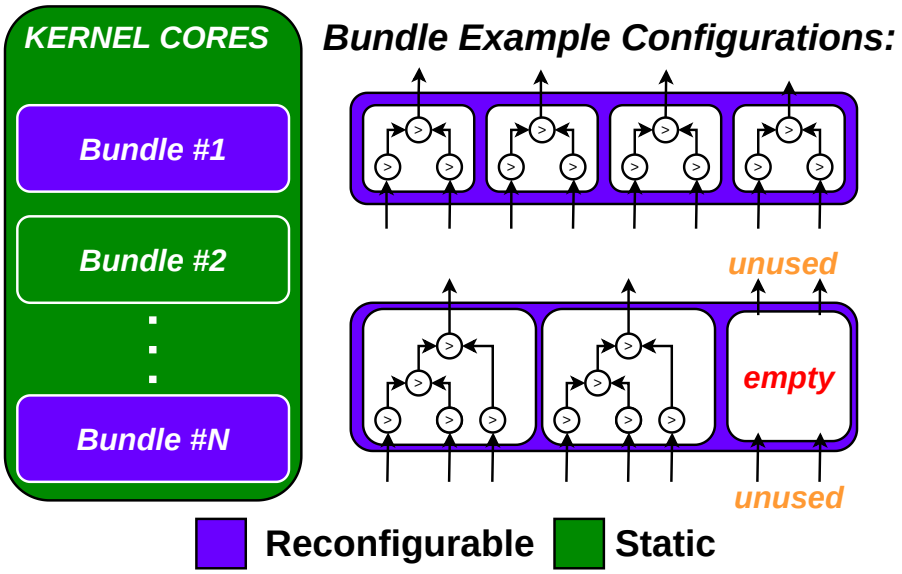


Figure 5.10 – Abstraction and Example Configuration of a single Bundle.

5.5 Temporary Layer Accumulator

As already discussed in Section 3.1, because of hardware constraints, it is not possible to calculate one output channel in a single iteration. This means that the temporary data has to be stored somehow and accumulated over all required iterations.
Figure 5.11 illustrates the approach used to retrieve actual output channel data. During the calculation of temporary output channels, the bit width precision of the quantized values is expanded from 8-bit to 24-bit inside the *Routed Kernel Cores* unit to prevent potential over- or underflows of the values during computation.
When the incoming data is streamed into the *Temporary Layer Accumulator* in its non-requantized form with 24 bits of precision, a read transaction is triggered on the FIFO in the first cycle. Following that, the data already residing in the FIFO, as well as the delayed pixel stream, are combined and routed back into the buffer, where the data is written back.
To ensure that the BRAM FIFO works as expected, it is initialized as *full* during reset ($is_full = 1$), and the $depth$ is set to the size of one output channel, so that all data is accumulated correctly (compare Section 5.2).
When the final accumulation phase is being performed ($retrieve_output = 1$), the data is streamed through a clipping unit that uses the shift amount provided by the *Configuration Unit* (compare Section 3.2) to map all values back to an 8-bit integer. Following that, the resulting 8-bit streams are provided to the corresponding output ports, so they can be packed by the *AXIS Packer Unit* (compare Section 5.1.4).

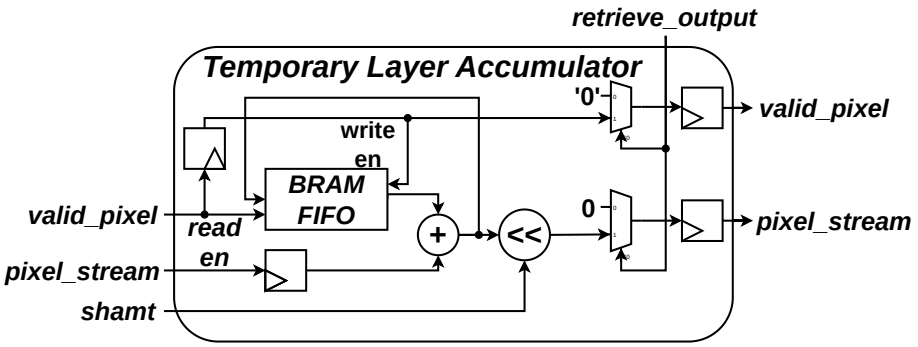


Figure 5.11 – Schematic of the *Temporary Layer Accumulator*.

One concern with such a design is determining how large the BRAM FIFO needs to be for this approach to work. When considering the AlexNet architecture (see Chapter 6), the largest output is a 55×55 image. Considering that every pixel has a size of 24 bits before clipping, this would lead to a maximum memory consumption of $\approx 8.86kB$, which is negligible.
Because of that, it is possible to place the unit statically on the PL and provide it only once, since, from a design perspective, it currently does not make sense to compute multiple temporary output channels. At that point, the existing resources could be used just to compute a single output channel in the first place. The unit also skips the accumulation if a pooling phase is performed. This is checked by the *Control Unit* and the corresponding control signal *retrieve output*, which is provided to the *Temporary Layer Accumulator*.
Another approach was also considered, where the temporary data would have been directly streamed over AXI-Stream to the software driver, but this would have led to many unnecessary write transactions, which are circumvented with only a relatively small amount of extra logic and memory.

5.6 Block & Software Integration

With all submodules in place, the only thing missing is the integration of the Top Level together with the software. Figure 5.12 illustrates the block design used to achieve this goal.
Because the Top Level requires two AXI-Stream slave interfaces, one for the pixel data and one for the weight data, the unit instantiates two different *AXI Direct Memory Access* IP cores [29]. Both IP cores are configured to use a read channel, but only the DMA IP core used for the pixel data is also configured with an additional write channel to receive the outgoing channel data from the Top Level.

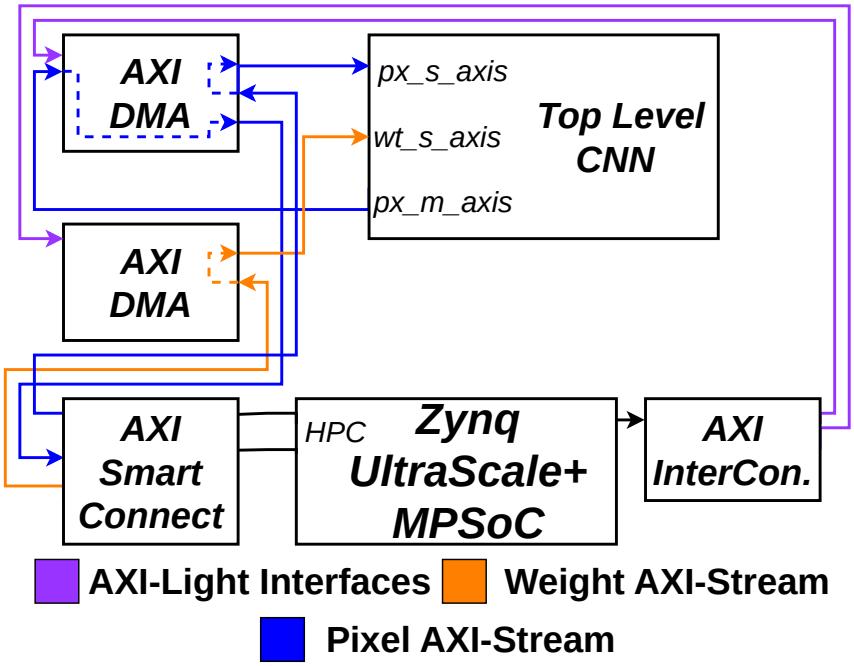


Figure 5.12 – Block Design of the Top Level.

The DMA IP core used for the weight data is configured with a buffer length register width of 9 bits. This is because the largest weight packet in the AlexNet architecture, which will be used to prototype the design (compare Chapter 6), is 400 Bytes long. This is the case for the second convolution phase computing 16 concurrent input channels. The DMA IP core responsible for the pixel data is configured to use a bit width of 19 bits, because the largest packet the unit needs to receive is provided by the first pooling layer, which needs to send $55^2 \cdot 96 = 290400$ Bytes in one transaction.

The Stream and Memory-Mapped Data Widths are set to 128 bits to maximize throughput and minimize the number of transactions. The Burst Size is configured to be 16, which is due to the Zynq UltraScale+ MPSoC being AXI-3 compliant and therefore would not show performance boosts when increasing the Burst Size [30].

To further optimize the design's performance, the processor is configured to use two cache-coherent AXI-Slave interfaces, both of which are used for sending weight and incoming channel data as well as for retrieving outgoing channel data. The arbitration is handled by the *AXI SmartConnect* IP core.

Finally, the AXI-Lite commands are sent by the PS over a single AXI-Master interface line, which is multiplexed via an *AXI Interconnect* to both DMA IP cores.

With this block design in place, the Top Level can be fully integrated into the system, using the *Interrupt System* provided by the DMA IP cores to control when the software driver should send or receive data consumed or provided by the design.

Chapter 6

Evaluation

Table 6.1 – Hardware Resources of the PL present on the Ultra96-v2 [31].

Programmable Logic (PL):	Zynq Ultrascale+ MPSoC
CLB LUTs	70.560
CLB Flip-FLops	141.120
Distributed RAM	1.8 Mb
Block RAM Tiles	216
DSP Slices	360

Before jumping to the actual benchmarking results, the environment in which the benchmarks are conducted needs to be established. The evaluation platform on which the design will be prototyped is the *Ultra96-v2* board [31], an ARM-based Zynq UltraScale+ MPSoC. The PS consists of a Quad-Core *ARM Cortex-A53*, and the PL includes the hardware resources listed in Table 6.1.
First, the design will be evaluated for scalability, focusing on the architecture's hardware utilization. For that, Section 6.1.1 will explore how the hardware itself, excluding additional IP cores, scales in terms of hardware resources and theoretically achievable frequencies, discussing which parameters are flexible and which are currently static.
This is followed up in Section 6.1.2, which showcases an exemplary integration of the design and discusses the hierarchical distribution of hardware resources.
For all synthesis and implementation benchmarks, the tool-chain provided by *Vivado 2023.2* [32] was used. The theoretical implementations were achieved using the *Out-Of-Context* mode, with all ports assigned a delay tolerance of 10% compared to the virtual clock frequency. Both the theoretical and actual implementation benchmarks were performed using the standard *Vivado* synthesis flow.

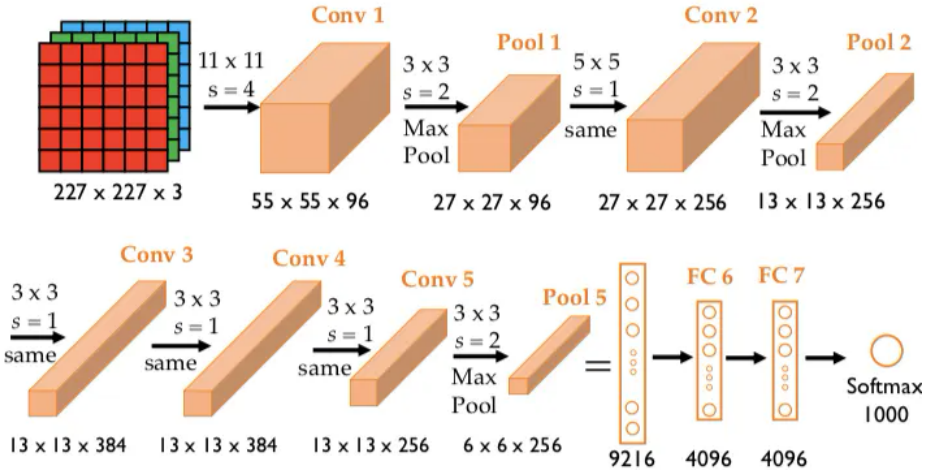


Figure 6.1 – Overview of the *AlexNet* architecture [22, 33].

After presenting the design's scalability, the performance and, therefore, functionality need to be investigated as well. As already hinted at in Section 4.2, the flexibility of the design is illustrated by instantiating and benchmarking each convolution and pooling layer of the *AlexNet* CNN architecture [22] in two different configurations for each layer, respectively. One version is faster but more hardware-intensive, and the other is slower but less hardware-demanding. The CNN architecture in question is depicted in Figure 6.1.

As it will be made clearer during the theoretical hardware utilization benchmark, only the *Kernel Size* and the concurrent *Channel Amount* change resource utilization. As a result, certain hardware configurations can be reused to implement different CNN layers. The necessary configurations, along with their indexes (compare Figure 6.1), are listed in Table 6.2.

The table also states the frequencies at which the configurations could be implemented, being either 250 MHz or 300 MHz, depending on the routability of the implementations at hand.

For all future benchmarks referencing these configurations, the corresponding identifier will be used to reference these specific layer configurations. If a hardware configuration is reused for multiple layers, the identifier will be enumerated to mark the relative order of these layers. All parameter modifications of preexisting *AlexNet* layers are explicitly listed for each individual benchmark.

The design, implemented as a bare-metal application on the *Ultra96-v2* evaluation platform, will further be compared to the previously mentioned single-threaded C/C++ reference implementation (compare Section 4.2), running on an *Intel® Core™ i7-9750HF* [34], an CISC processor with a base frequency of 2.60 GHz and a maximum turbo frequency of 4.50 GHz to see how the embedded design holds up against an x86 architecture.

Table 6.2 – *AlexNet* Hardware Configurations.

Layer Identifier	Input Channel	Output Channel	Input Width	Output Width	Channel Amount	Frequency (MHz)
Conv 11×11	3	96	227	55	Slow: 1 Fast: 3	300 250
Conv 5×5	96	256	27	27	Slow: 8 Fast: 16	300 250
Conv 3×3	256 384 384	384 384 256	13	13	Slow: 8 Fast: 16	300 250
Pool 3×3	96 256 256	96 256 256	55 27 13	27 13 6	Slow: 8 Fast: 16	300

6.1 Resource Utilization

6.1.1 Theoretical Hardware Utilization

Most of the parameters making up a CNN layer can actually be sent by the software to program the hardware (compare Section 3.2). Unfortunately, this is restricted to some extent, meaning that certain parameters must be hardwired, at least with an upper bound. The main parameters that are not programmable and therefore responsible for increasing hardware utilization are the *Kernel Size* and *Channel Amount*. When considering that increasing these parameters also increases the number of multipliers, adders, or comparators needed, it is no surprise that they dictate the hardware resources.

Figure 6.2 showcases how the resource demand scales when increasing the concurrent number of channels the accelerator needs to calculate. Currently, the design is limited to the maximum bus width achievable on the *Ultra96-v2* platform, being 16 Bytes. Because of that, various $Channel\ Amount \in [1, 16]$ were investigated on how the LUT and Flip-Flop amount, scaled by the left Y-Axis, and how the BRAM and DSP amount, scaled by the right Y-Axis, changed when increasing said parameter. The benchmark was conducted with a static *Kernel Size* of three.

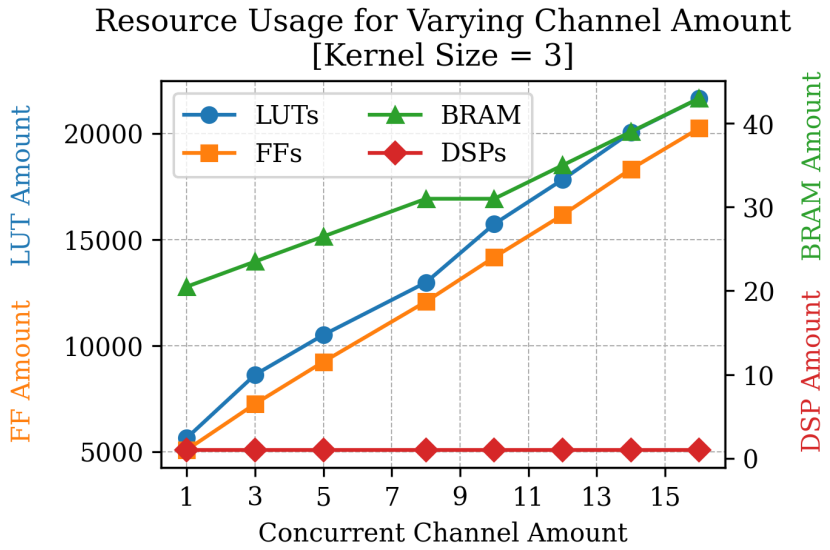


Figure 6.2 – Resource Utilization of a Convolution for varying *Channel Amount*.

As shown in the plot, the LUT and FF amount scale linearly with increasing channel count. This is to be expected, considering that when the concurrent number of channels to be calculated is, for example, doubled, twice as many cores will be needed to compute the temporary layer. Because of multiplier being the biggest source of hardware (compare Section 6.1.2), it is no surprise that the amount of hardware utilized is nearly doubled. This can be seen, for example, when comparing the LUT and FF amount between 8 and 16 concurrent channels.

The gradient of the LUT amount also shows a slight decrease for both 8 and 16. This could be due to the fact that the unit does not instantiate an *AXIS Data Width Converter* (compare Section 5.1.2) for channel amounts that are bases of 2. The slight decrease could also be pinpointed to more optimal tree structures. Whatever the root cause may be, it is to be expected to see a slight decrease in hardware resource growth for these specific parameters.

For the BRAMs linear increase the same explanation as with the LUTs and FFs can be given. What is more interesting is the DSP count, which is always 1. This is because only one DSP is instantiated throughout the design. In more detail, it implements the multiplication performed by the *Weight Unit*, which needs to calculate the amount of data it will receive (compare Section 5.3).

Because of the bit width of the configuration values (compare Section 3.2), the multiplication produces a 32-bit result, which the synthesis tool maps to a DSP. This mapping was kept because the *Weight Unit* is a static design, meaning that no rerouting to a DSP device would be needed.

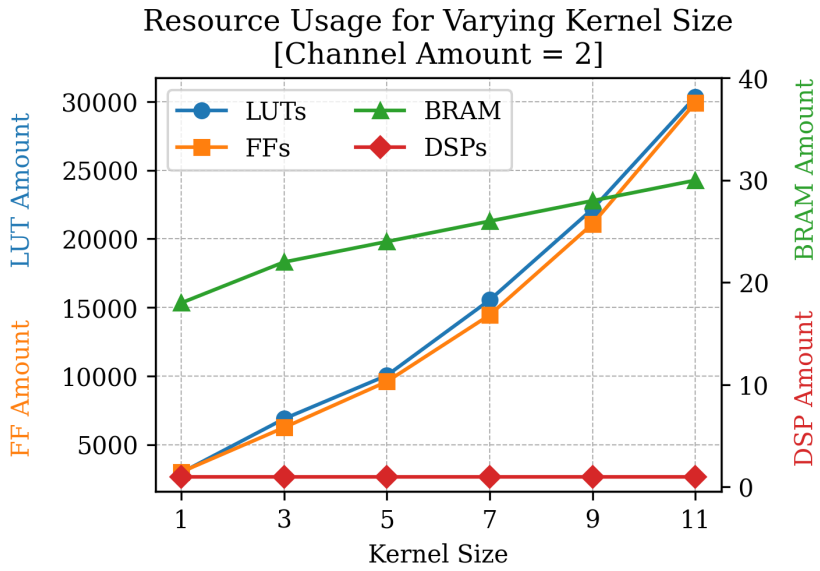


Figure 6.3 – Resource Utilization of a Convolution for varying *Kernel Size*.

Unlike the variation of the concurrent *Channel Amount*, the increase in *Kernel Size* scales the design differently. Figure 6.3 plots hardware usage for changing the size of the kernel. The *Channel Amount* is fixated at two.

This time, the LUT and FF amount do not scale linearly, but rather quadratically. This can be traced back to the *Kernel Size* increasing quadratically, meaning that, for a convolution phase, e.g., going from a *Kernel Size* 3 to 5 would require 25 multipliers instead of 9 for a single channel.

The increase in BRAM, however, stays linear. The reason is that, for each increase in the *Kernel Size*, the design requires only one additional *Line Buffer* per channel to store an extra image row. The DSP amount is again stuck at 1 for the same reasons mentioned for the previous plot.

Another interesting observation can be made when comparing both factors in terms of the theoretical frequencies achievable. The decrease in frequency resulting from increased hardware utilization is shown in Figure 6.4. In both figures, the correlation between an increase in hardware resources and a decrease in maximum frequency is apparent when considering the upper bounds of both parameters.

The frequencies of both diagrams can be directly compared to some extent, provided that both plots start at $\approx 360\ MHz$ in their smallest configurations. For a high *Kernel Size*, the frequency drops relatively low to $\approx 310\ MHz$, whereas the maximum *Channel Amount* still manages to maintain a frequency of $\approx 330\ MHz$. This is again traceable to the fact that the *Kernel Size* has a steeper increase in required resources.

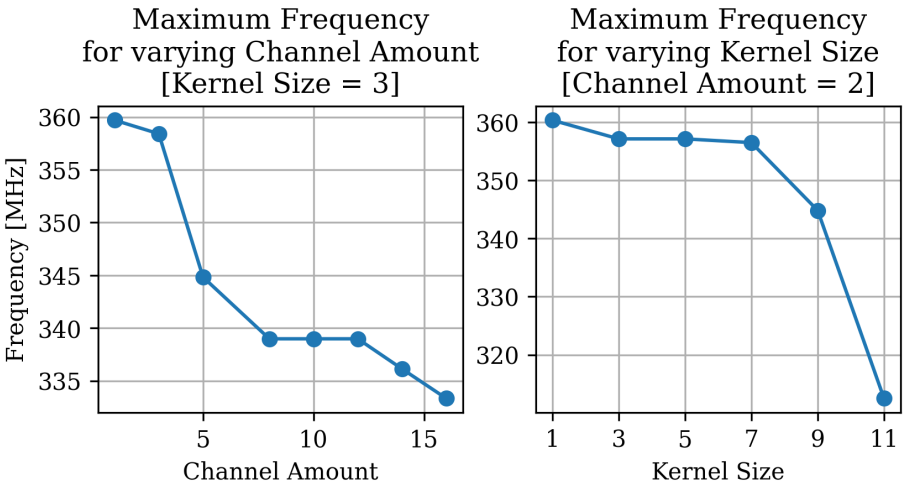


Figure 6.4 – Theoretical achievable Frequencies of Convolution Configurations.

What is interesting and somewhat unexpected is the gradients with which both plots are falling. The increase in concurrent channels has a much steeper decrease at the beginning, but levels off at a channel size of 8. Contrary to that, the frequencies for different smaller *Kernel Sizes* are fairly consistent, but start to drop sharply at a *Kernel Size* of 9 or higher.

The reason for this could be that the increase in the use of BRAM tiles is greater when increasing the number of concurrent channels than when increasing the kernel size. With that, the routing tool may not find better placements to increase frequency. This, however, is just a hypothesis; more research would be needed to evaluate this. Currently, the *Kernel Size* and the *Channel Amount* are the only attributes that scale the hardware. Other CNN-related attributes, such as the number of *Input Channels*, *Output Channels*, or the *Input Image Length*, are all controlled by the software (compare Section 6.2.2). Other attributes, such as the AXI-Stream bus width of the weight data, must be static because of the configuration string requiring a single 128-bit instruction. Combined with the current abstraction that uses the same generic for all AXI-Stream interfaces, this results in all AXI-Stream interfaces currently being static. As a result, the bit width is also static, as it is heavily adapted to the bus size and the corresponding *AXIS Packer Unit* and the *AXIS Unpacker Unit*. Nonetheless, throughout this theoretical hardware benchmark, a few surprising discoveries were made, but most results were as expected and showed promising scalability of the proposed design.

6.1.2 Practical Hardware Integration

Table 6.3 – Hierarchical Resource Consumption of Implemented *Conv* (3 × 3).

Instance (Slow Conv (3 × 3)**)**	**Total LUTs**	**FFs**	**BRAM**
Top_Level_wrapper	19989 (28.33%)	23126 (16.39%)	38.5 (17.84%)
axi_dma_0	2036 (2.89%)	3147 (2.23%)	5 (2.31%)
axi_dma_1	639 (0.91%)	961 (0.68%)	2.5 (1.16%)
axi_smc	3784 (5.36%)	6340 (4.49%)	0 (0.00%)
ps8_0_axi_periph	1314 (1.86%)	1446 (1.02%)	0 (0.00%)
rst_ps8_0_99M	15 (0.02%)	34 (0.02%)	0 (0.00%)
top_level_cnn_wrapper_0	12205 (17.30%)	11198 (7.94%)	31 (14.15%)
(top_level_cnn_wrapper_0)	792 (1.12%)	0 (0.00%)	0 (0.00%)
U0	11419 (16.18%)	11198 (7.94%)	31 (14.15%)
top_level_cnn_inst	11419 (16.18%)	11198 (7.94%)	31 (14.15%)
(top_level_cnn_inst)	0 (0.00%)	48 (0.03%)	0 (0.00%)
axis_2_configurable_slider_wrapper_inst	2801 (3.97%)	3031 (2.15%)	20 (9.26%)
(axis_2_configurable_slider_wrapper_inst)	0 (0.00%)	2 (0.01%)	0 (0.00%)
axis_2_pxs_inst	854 (1.21%)	1055 (0.75%)	8 (3.70%)
configurable_slider_inst	1948 (2.76%)	1974 (1.40%)	12 (5.56%)
control_unit_axis_pipeline_step	326 (0.46%)	292 (0.21%)	0 (0.00%)
control_unit_inst	876 (1.24%)	1505 (1.07%)	0 (0.00%)
(control_unit_inst)	59 (0.08%)	43 (0.03%)	0 (0.00%)
config_unit_inst	263 (0.37%)	239 (0.17%)	0 (0.00%)
weight_unit_inst	555 (0.79%)	1223 (0.87%)	0 (0.00%)
pxs_2_axis_inst	937 (1.33%)	724 (0.51%)	8 (3.70%)
(pxs_2_axis_inst)	0 (0.00%)	21 (0.01%)	0 (0.00%)
axis_pipeline_step_inst	78 (0.11%)	174 (0.12%)	0 (0.00%)
bram_fifo_array_inst	844 (1.20%)	512 (0.36%)	8 (3.70%)
pxs_2_axis_demux_inst	15 (0.02%)	17 (0.01%)	0 (0.00%)
routed_kernel_cores_inst	6388 (9.05%)	5474 (3.88%)	0 (0.00%)
KERNEL_CORES_INST	6228 (8.83%)	4266 (3.02%)	0 (0.00%)
ROUTING_CORES_INST	160 (0.23%)	1208 (0.86%)	0 (0.00%)
temp_layer_accumulator_inst	120 (0.17%)	124 (0.09%)	3 (1.39%)

Now that the architecture's resource consumption has been extensively covered, the actual integrated design (compare Section 5.6) needs to be investigated as well. Table 6.3 showcases the hierarchical resource distribution of a Convolution Layer with a *Kernel Size* of three and a *Channel Amount* of eight (compare Table 6.2). This configuration was specifically chosen because of its central role in the *AlexNet* architecture, serving as a convolutional phase that is used three times throughout the CNN architecture.

Before jumping into the conclusions, however, it should be noted that, for synthesis and implementation, *Vivado* performs a *Cross-Boundary Optimization*, which means the resource distributions might not be completely accurate. Nevertheless, for this particular design, these optimizations are negligible.

From the table, it is apparent that the additional IP cores used to integrate the main accelerator design take up ≈ 11% of the PL present on the *Ultra96-v2* board. This part of the hardware distribution is consistent across all design configurations. For smaller configurations, this hardware demand could even be reduced, but it was decided against this so that, when the design is partially reconfigured, it can work with larger layers more easily.
Having said that, an extra resource demand of ≈ 11% is definitely more than acceptable on the *Ultra96-v2* platform. On more resource-constrained devices, only utilizing one *AXI DMA* could be a way to cut hardware costs, but this would also mean that the *Top Level* would need to provide additional multiplexing of the AXI-Streams. Another way to reduce hardware costs would be to design the system to support fewer concurrent channels, as shown in Section 6.1.1. From the table, it is apparent that the *Routed Kernel Cores* unit consumes most of the hardware needed by the design, with ≈ 9% in this configuration. This is again due to the various multipliers and adders present in the core. With the *Channel Amount* decreased, this would also reduce the costs of the *Configurable Slider*, especially in terms of BRAM utilization, which in the current example take up almost $\frac{1}{3}$ of all BRAM tiles used throughout the entire design.
One metric the table does not cover is the amount of DSP used throughout the design. As previously mentioned, only one DSP is currently used inside the design, mainly in the *Weight Unit* (compare Section 6.1.1).
With these discoveries in mind, along with the resolutions of the previous section, it is safe to say that the architecture is flexible enough to be adapted for more resource-constrained devices, yet flexible enough that larger FPGA chips can benefit from their larger PL regions by adjusting the number of concurrent channels calculated.

6.2 Performance Benchmarking

6.2.1 Performance Comparison of Hardware and Software

To get a general understanding of the design's performance, Figure 6.5 plots the latencies of all *AlexNet* convolution and pooling phases on a logarithmic scale for three versions each: the two previously mentioned hardware configurations and the software reference implementation.
For the hardware configurations, all layers were computed 10,000 times. From these samples, the means as well as the standard deviations were derived. For the software, each layer was computed 1,000 times to derive these values.

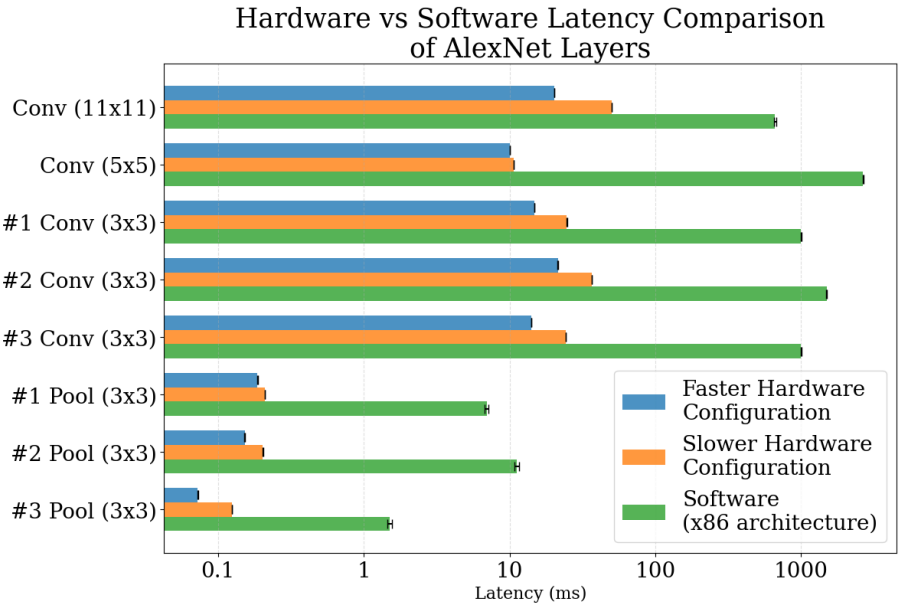


Figure 6.5 – Latency Comparison of all *AlexNet* CNN Layers.

From the plot, it is apparent that the hardware designs easily outperform the C/C++ reference implementation running on an x86 architecture. Especially when looking at, e.g., the *Conv* (5×5) layer, a speed-up factor of roughly $\times 264$ is observed with the faster hardware configuration compared to the software reference.
Interestingly, the faster hardware configuration is often not much faster than the slower one, as is the case, e.g., for the *Conv* (5×5) layer. This is very likely due to the slower configuration being able to run at a higher frequency of 300 MHz instead of the 250 MHz the faster configuration is running at. From this observation, it is fair to say that the longer the design runs for an individual layer, the more hardware-intensive solutions might not be the optimal answer all the time, even if they are placeable.
Another observation that is not immediately visible from the graph is the low *Standard Deviation (STD)* the design has over all iterations. For all hardware configurations, the STD stays below $\approx 16\ \mu s$, whereas the software has much larger spikes of up to $\approx 11\ ms$ depending on the layer.
Although the hardware designs easily beat the software reference, it is important to note that these benchmarks should be considered as best-case performances. The reason is that the software driver is currently not performing the data movements that will be necessary when computing actual layer data. Future work would need to investigate the impact of data movements performed by the software on the overall performance (compare Chapter 7).

6.2.2 Performance Comparison of Varying Parameters

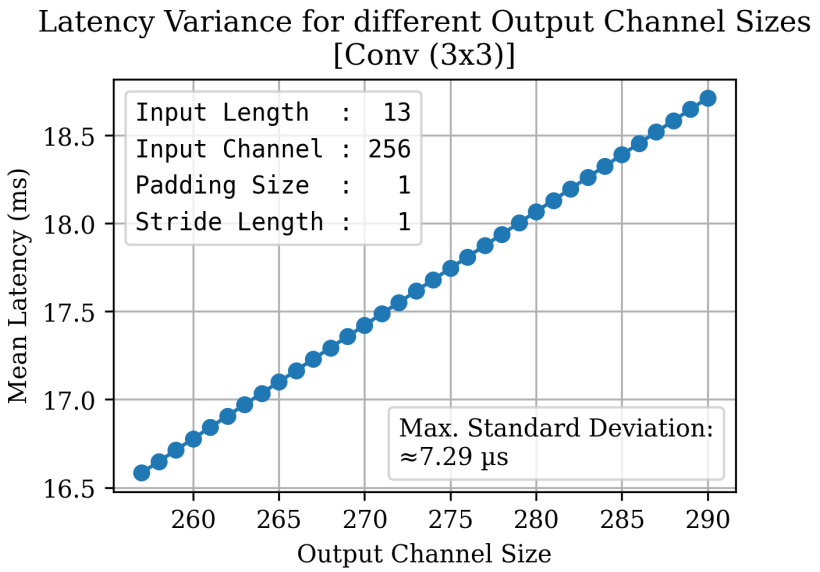


Figure 6.6 – Latency Variance for different *Output Channel Sizes*.

Now that we have received a rough understanding of how the design scales for different-sized layers, we still need to investigate the impact on the performance of all the parameters controllable by the software. In the following, all benchmarks will be performed on a Convolution Layer with *Kernel Size* set to three, with eight concurrent channels.

Figure 6.6 illustrates how the design is impacted if the output channel amount is increased. Unsurprisingly, because all output channels are computed sequentially (compare Chapter 3), the design shows a linear increase in latency with an overall small deviation of at most $\approx 7.29\ \mu s$.

Similarly to the output channel amount, the same benchmark can be performed for increasing the input channel amount. Unlike the output channel benchmark, however, this value is limited so that it must be a multiple of the number of channels computed concurrently. In our case, this means the input channel count must be a multiple of eight.

Figure 6.7 illustrates the performance variance achieved when increasing the input channel amount. Increasing this parameter adds a fixed number of iterations to the accelerator's runtime. Therefore, it is no surprise that the increase in latency is linear and fairly consistent, with a maximum STD of $\approx 18.33\ \mu s$.

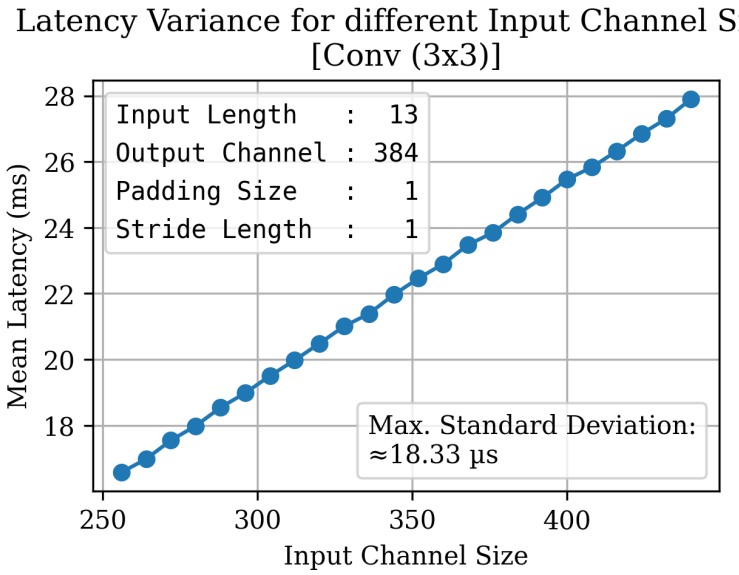


Figure 6.7 – Latency Variance for different *Input Channel Sizes*.

Not only can the channel amounts be configured, but other parameters, such as the image dimensions, can also be programmed. Figure 6.8 illustrates the impact on performance when increasing the sizes of images sent by the software. Compared to the previous benchmarks, the latency increase here is quadratic, as expected, since the number of pixels the design needs to process scales with $input_size^2$.
It is important to note that hardware constraints restrict this parameter. For one, the FIFO sizes of the *Line Buffer* structures need to be large enough to store entire image rows. Currently, these FIFOs support a maximum capacity of 512 elements, but this can be increased as needed.
Also, either the buffer length register width of an *AXI DMA Unit* must be large enough to stream the image data in a single transaction, or the software driver must split up the packages into smaller chunks.

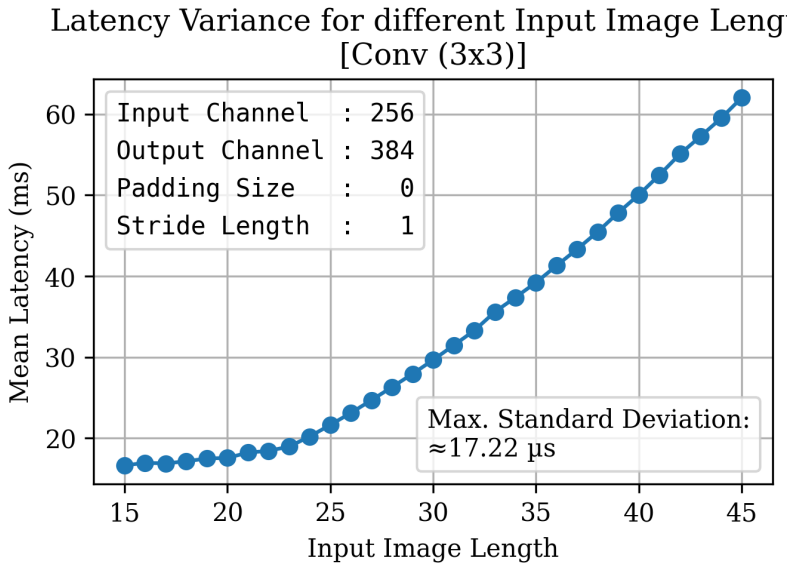


Figure 6.8 – Latency Variance for different *Input Image Length*.

Another parameter that should behave fairly similarly to the input image length is the *Padding Size*, because it has a similar quadratic effect on the number of pixels the unit needs to process.

Figure 6.9 illustrates the benchmark for increasing the padding amount of an input image. As expected, the latency increase is quadratic, similarly to the previous benchmark. What is different, however, are the gradients in which the benchmarks are rising. The latency increases much more steeply with increasing padding size than with increasing image input length.

This might seem quite counterintuitive, because the padding data does not need to be streamed into the design. Nevertheless, when considering that much more data has to be processed by the design when increasing the padding compared to increasing the image input length, it is reasonable that the latency increase for padding is that severe.

This discovery, together with the previous benchmarks, also reveals another fact. The performance of the design is mainly limited by the fact, that it can only process one pixel for each channel per clock cycle. This also seems plausible, given that the design receives 16 Bytes of data per valid AXI-Stream transaction (compare Section 5.6), meaning it receives more data than it can process in a single cycle. Therefore, it would be a relatively rare occurrence for the design to run empty.

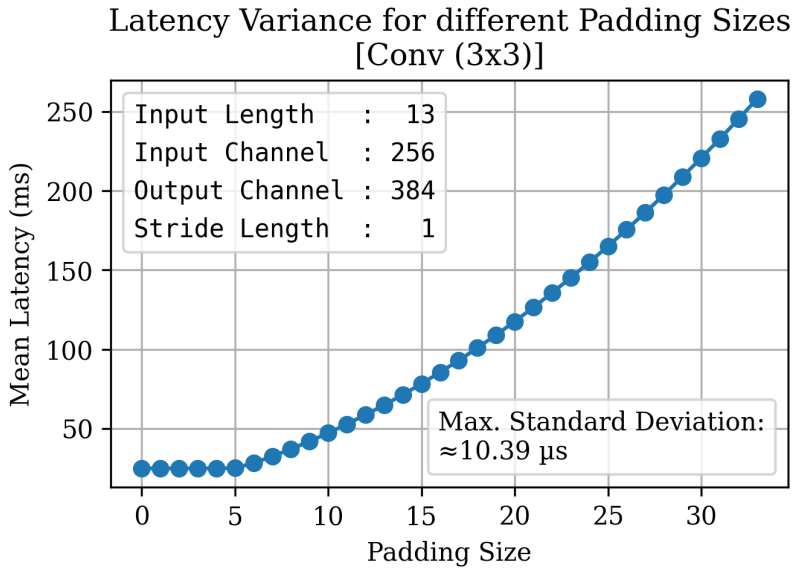


Figure 6.9 – Latency Variance for different *Padding Sizes*.

That conclusion is further emphasized by the last parameter that the software can program: The *Stride Length*.
Figure 6.10 shows the performance impact of increasing the *Stride Length*. In the plot, it is apparent that although the performance is not static, all datapoints are fairly close together, only being separated by at most 100 μs. Similarly to the *Padding Size*, this result might seem implausible at first glance, but it makes sense when considering that, even when the *Stride Length* is increased, all pixels still need to be processed by the design. As a result, no noticeable performance gain can be achieved. The only part of the design that could introduce performance variance, which is very likely also the root cause of the minor spikes in the plot, is the *AXIS Data Packer* and the retrieval duration of all packages.
However, taking all previous plots into account, it is safe to say that the design is throttled mainly by the performance of the *Line Buffer* structures.

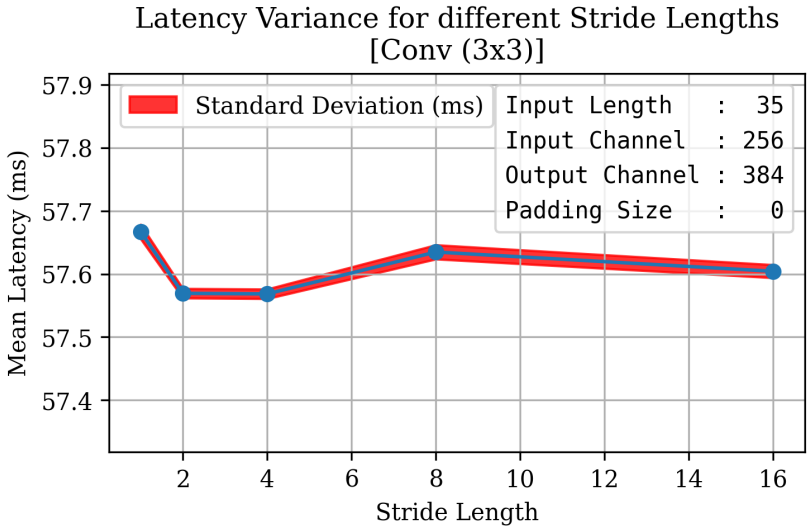


Figure 6.10 – Latency Variance for different *Stride Lengths*.

Chapter 7

Conclusion

The use of ASIC- and FPGA-based implementations revolutionized how *Convolutional Neural Networks (CNN)* can be deployed in more and more resource-constrained, real-time-dependent systems.
This work showcased new design abstractions to enable more dynamic, partially reconfigurable CNN architectures in the FPGA domain. It illustrated the research needed to verify practicality, while also demonstrating the design and tooling required to bring the concept to life.
The architecture has been implemented across various CNN layers to benchmark the design's scalability in terms of hardware utilization and performance. The work presented achieved a best-case speedup factor of up to $\times 264$ for single-layer inference compared to a C/C++ reference implementation running on an x86 architecture.
All in all, the work offered a glimpse into the interesting yet largely unexplored topic of partial reconfiguration. Despite that, many questions remained unanswered and were left as *Future Work*.
The most interesting aspect of benchmarking would be to measure the actual reconfiguration times of the architecture and compare several FPGA architectures to assess their differences in bandwidth and, therefore, in actual reconfiguration times. Another closely related topic to the reconfiguration would be to measure the data movements that the software driver must perform to investigate how the design's performance holds up. This would also leave room to explore the performance of this design across entirely different CNN architectures to gather more data in this regard and better benchmark overall performance.

Besides conducting more benchmarking, the design and tooling could be further improved. For one, the design could be expanded to include subsequent pooling phases directly following convolution phases to increase throughput and thereby reduce reconfiguration cycles. Apart from potential expansion of the design, the Python-based code-generation framework could also be further developed to support a multitude of features, such as automatic static tree generation, if partial reconfiguration is not desired. The tooling's logical routing could also be improved to route the cores more efficiently. Further, the code generation could be expanded to generate the necessary TCL files for the synthesis and implementation of the design. Besides the code-generation framework, the training could also be improved to converge faster. With that, not only can accuracy be improved, but new weight data can be retrieved much more quickly.
With all the results presented and all the possibilities for *Future Work* left in this area of partial reconfiguration, more interesting discoveries can most definitely be made when continuing research in this yet only partially discovered field.

List of Figures

1.1 General Accelerator Structure. 2

2.1 Block Design of Google's *Tensor Processing Unit* [13]. 5
2.2 Example of the *FINNs* end-to-end flow [14]. 6
2.3 *Macro Block* based design with dynamic interconnections. [10] 7
2.4 Schematic of the *Kernel Computation Unit* [21]. 8

3.1 Proposed Accelerator Design for generic CNN Layers. 12
3.2 Simplified Moore FSM representation of the Control Logic. 14
3.3 Configuration Instruction for programming the Hardware. 15

4.1 Snippet of example YAML Configuration. 18

5.1 Schematic of skid buffer for pipelining AXIS Interfaces [26]. 22
5.2 Simplified data flow of a 4 Byte to 3 Byte *AXIS Downsizer* [28]. 23
5.3 Architecture Design of the *AXIS Unpacker Unit*. 24
5.4 Write enable hazards with read multiplexing (single channel). 25
5.5 Read enable hazard control for single BRAM FIFO. 26
5.6 Architecture Design of the *AXIS Packer Unit*. 27
5.7 Schematic of a partially reconfigurable *Line Buffer* Structure. 28
5.8 Schematic of the *Weight Unit*. 29
5.9 Block Design of Adder/Comparator Tree Structure. 31
5.10 Abstraction and Example Configuration of a single Bundle. 32
5.11 Schematic of the *Temporary Layer Accumulator*. 33
5.12 Block Design of the Top Level. 34

6.1 Overview of the *AlexNet* architecture [22, 33]. 36
6.2 Resource Utilization of a Convolution for varying *Channel Amount*. . 38
6.3 Resource Utilization of a Convolution for varying *Kernel Size*. 39
6.4 Theoretical achievable Frequencies of Convolution Configurations. . 40
6.5 Latency Comparison of all *AlexNet* CNN Layers. 43

6.6 Latency Variance for different *Output Channel Sizes*. 44
6.7 Latency Variance for different *Input Channel Sizes*. 45
6.8 Latency Variance for different *Input Image Length*. 46
6.9 Latency Variance for different *Padding Sizes*. 47
6.10 Latency Variance for different *Stride Lengths*. 48

List of Tables

4.1 Comparison between different *CIFAR-10* inference approaches. 19

6.1 Hardware Resources of the PL present on the Ultra96-v2 [31]. 35
6.2 *AlexNet* Hardware Configurations. 37
6.3 Hierarchical Resource Consumption of Implemented *Conv* (3×3). . . 41

Bibliography

[1] I. Chaudhary and B. P. Lohani, "A Study on Recent Advancements and Innovations in Convolutional Neural Network," in *2024 7th International Conference on Contemporary Computing and Informatics (IC3I)*, vol. 7, 2024, pp. 1014–1019 (cit. on p. 1).

[2] T.-D. Do, M.-T. Duong, Q.-V. Dang, and M.-H. Le, "Real-Time Self-Driving Car Navigation Using Deep Neural Network," in *2018 4th International Conference on Green Technology and Sustainable Development (GTSD)*, 2018, pp. 7–12 (cit. on p. 1).

[3] I. D. Mienye, T. G. Swart, G. Obaido, M. Jordan, and P. Ilono, "Deep Convolutional Neural Networks in Medical Image Analysis: A Review," *Information*, vol. 16, no. 3, 2025. [Online]. Available: `https://www.mdpi.com/2078-2489/16/3/195` (cit. on p. 1).

[4] M. R. Z. Chowdhury, A. Seum, M. R. Talukder, R. A. Amin, F. S. Hossain, and R. Obermaisser, "Towards Next-Generation FPGA-Accelerated Vision-Based Autonomous Driving: A Comprehensive Review," *Signals*, vol. 6, no. 4, 2025. [Online]. Available: `https://www.mdpi.com/2624-6120/6/4/53` (cit. on p. 1).

[5] D. Moolchandani, A. Kumar, and S. R. Sarangi, "Accelerating CNN Inference on ASICs: A Survey," *Journal of Systems Architecture*, vol. 113, p. 101 887, 2021. [Online]. Available: `https://www.sciencedirect.com/science/article/pii/S1383762120301612` (cit. on p. 1).

[6] T. von Sydow, B. Neumann, H. Blume, and T. G. Noll, "Quantitative Analysis of Embedded FPGA-Architectures for Arithmetic," in *IEEE 17th International Conference on Application-specific Systems, Architectures and Processors (ASAP'06)*, 2006, pp. 125–131 (cit. on p. 1).

[7] S. Y. Neyaz, I. Saxena, N. Alam, and S. A. Rahman, "FPGA and ASIC Implementation and Comparison of Multipliers," in *2020 International Symposium on Devices, Circuits and Systems (ISDCS)*, 2020, pp. 1–4 (cit. on p. 1).

[8] J. Xu, "Comparison and Application of FPGA and ASIC in Digital System Design," in *Proceedings of the International Conference on Decision Science & Management*, ser. ICDSM '24, Association for Computing Machinery, 2024, pp. 216–220. [Online]. Available: https://doi.org/10.1145/3686081.3686117 (cit. on p. 1).

[9] *Documentation of Xilinx's Dynamic Function eXchange*, Accessed: 2025-11-13. [Online]. Available: https://docs.amd.com/r/en-US/ug909-vivado-partial-reconfiguration/ (cit. on pp. 2, 7).

[10] H. Irmak, D. Ziener, and N. Alachiotis, "Increasing Flexibility of FPGA-based CNN Accelerators with Dynamic Partial Reconfiguration," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, 2021, pp. 306–311 (cit. on pp. 2, 7).

[11] M. Ghasemi and Y. Yang, "A Novel Design of Adaptive and Hierarchical Convolutional Neural Networks using Partial Reconfiguration on FPGA," Sep. 2019, pp. 1–7 (cit. on pp. 2, 7).

[12] J. Olivito, F. Serrano, J. Clemente, H. Mecha, and J. Resano, "Analysis of the reconfiguration latency and energy overheads for a Xilinx Virtex-5 FPGA," *IET Computers & Digital Techniques*, vol. 12, Jan. 2018 (cit. on p. 2).

[13] N. P. Jouppi et al., *In-datacenter Performance Analysis of a Tensor Processing Unit*, 2017. arXiv: 1704.04760 [cs.AR]. [Online]. Available: https://arxiv.org/abs/1704.04760 (cit. on p. 5).

[14] *Documentation of FINN*, Accessed: 2025-11-10. [Online]. Available: https://xilinx.github.io/finn/ (cit. on p. 6).

[15] S. Ghaffari and S. Sharifian, "Fpga-based convolutional neural network accelerator design using high level synthesize," in *2016 2nd International Conference of Signal Processing and Intelligent Systems (ICSPIS)*, 2016, pp. 1–6 (cit. on p. 6).

[16] *Documentation of Brevitas*, Accessed: 2025-11-09. [Online]. Available: https://xilinx.github.io/brevitas/ (cit. on pp. 6, 19).

[17] *Documentation of PyTorch*, Accessed: 2025-11-09. [Online]. Available: https://pytorch.org/docs/ (cit. on pp. 6, 19).

[18] Q. Ducasse, P. Cotret, and L. Lagadec, "Benchmarking quantized neural networks on fpgas with finn," Feb. 2021 (cit. on p. 6).

[19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778 (cit. on pp. 7, 19).

[20] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, pp. 2278–2324, Dec. 1998 (cit. on p. 7).

[21] M. Balajan and M. E. Gerber, "Hardware Acceleration of the LeNet-5-Architecture," *Rechnerarchitekturen für Deep-Learning Anwendungen*, 2025 (cit. on pp. 8, 11, 19, 29).

[22] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25, Curran Associates, Inc., 2012. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf (cit. on pp. 14, 19, 36).

[23] Y. Bi, B. Xue, and M. Zhang, "Evolving Deep Forest with Automatic Feature Extraction for Image Classification Using Genetic Programming," in Aug. 2020, pp. 3–18 (cit. on p. 19).

[24] E. Rex and X. Wang, "Efficient Quantization and Hardware Implementation of AlexNet on Resource-limited FPGAs," in *2023 IEEE 14th Annual Ubiquitous Computing, Electronics and Mobile Communication Conference (UEMCON)*, 2023, pp. 0399–0406 (cit. on pp. 19, 20).

[25] *Homepage of the CIFAR-10 Dataset*, Accessed: 2025-11-09. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html (cit. on p. 19).

[26] *Example Implementation of Skid Buffer*, Accessed: 2025-11-14. [Online]. Available: https://www.itdev.co.uk/blog/pipelining-axi-buses-registered-ready-signals (cit. on p. 22).

[27] S. Blank, "Adaptive Reconfigurable Wiring Architecture for Neural Networks on FPGAs," M.S. thesis, Friedrich-Alexander Universiät Erlangen-Nürnberg, 2022 (cit. on p. 22).

[28] *Example of AXIS Downsizer*, Accessed: 2025-11-09. [Online]. Available: https://github.com/suoto/fpga_cores/blob/master/src/axi_stream_width_converter.vhd (cit. on p. 23).

[29] *Documentation of Xilinx's DMA IP core*, Accessed: 2025-11-14. [Online]. Available: https://docs.amd.com/r/en-US/pg021_axi_dma/ (cit. on p. 33).

[30] *Zynq UltraScale+ MPSoC Embedded Design Methodology Guide*, Accessed: 2025-11-08. [Online]. Available: https://www.xilinx.com/support/documents/sw_manuals/ug1228-ultrafast-embedded-design-methodology-guide.pdf (cit. on p. 34).

[31] *Ultra96-v2 Hardware User's Guide*, Accessed: 2025-11-13. [Online]. Available: https://www.avnet.com/wcm/connect/b85b9556-0b2a-42b3-ad6a-8dcf3eac1ff9/Ultra96-V2-HW-User-Guide-v1_3.pdf?MOD=AJPERES&CACHEID=ROOTWORKSPACE.Z18_NA5A1I41L0ICD0ABNDMDDG0000-b85b9556-0b2a-42b3-ad6a-8dcf3eac1ff9-nDNP5R3?MOD=AJPERES&CACHEID=ROOTWORKSPACE.Z18_NA5A1I41L0ICD0ABNDMDDG0000-b85b9556-0b2a-42b3-ad6a-8dcf3eac1ff9-nDNP5R3 (cit. on p. 35).

[32] *Documentation of Vivado 2023.2*, Accessed: 2025-11-09. [Online]. Available: https://docs.amd.com/r/2023.2-English/ug949-vivado-design-methodology/Vivado-Design-Suite-User-and-Reference-Guides (cit. on p. 35).

[33] *Graphic of AlexNet-Architecture*, Accessed: 2025-11-11. [Online]. Available: https://miro.medium.com/v2/resize:fit:4800/format:webp/1*pmGLQBsuyvnQlDWGOfEQuw.png (cit. on p. 36).

[34] *Specification of Intel Core i7-9750HF*, Accessed: 2025-11-19. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/191046/intel-core-i79750hf-processor-12m-cache-up-to-4-50-ghz/specifications.html (cit. on p. 36).